2FA Bypass Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Two-Factor Authentication (2FA) Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Test only systems you own or are explicitly authorized to assess. Use dedicated test accounts and test-only verification codes. Never target another person's account or attempt to defeat their active MFA.*

■ Executive Summary

Two-factor authentication (2FA) is effective only when every protected action verifies the second factor and binds it to the correct user, session, and authentication attempt. The supplied five-page PDF lists common implementation weaknesses involving verification state, code reuse, backup codes, session expiry, response handling, and recovery flows.

■ Core Testing Methodology

1. **Map the login flow.** Document the initial credential step, MFA challenge, verification endpoint, and post-login destinations.
2. **Check identity binding.** With a test account, verify that the MFA challenge and verification code are bound to that account and the same login attempt.
3. **Check server-side enforcement.** Confirm that protected pages and APIs reject sessions that have completed only the password step.
4. **Review response handling.** Verify that the application decides authentication success on the server; client-side response edits must not grant access.
5. **Review code lifecycle.** Confirm that codes expire, are single-use, and are invalidated when a new challenge is issued where appropriate.
6. **Review backup codes.** Check that backup codes are high-entropy, single-use, rate-limited, and safely recoverable only through the intended flow.
7. **Review session lifecycle.** Confirm that enabling MFA, logging out, changing a password, or completing recovery invalidates or refreshes sessions according to policy.
8. **Review recovery and account changes.** Test email/password changes and MFA disable/reset flows using only controlled accounts.
9. **Check rate limits.** Verify bounded retries and monitoring for OTP and backup-code attempts without conducting brute-force activity.
10. **Document and retest.** Record the expected control, observed result, redacted evidence, and regression test.

■ High-Value Bug Bounty Tips & Edge Cases

- **Account-cookie trust:** The source describes a flow where a cookie identifies the account during MFA. Account identity should come from server-validated state, not a freely editable client value.
- **Clickjacking on MFA settings:** Protect pages that disable or reset MFA against framing where appropriate.
- **Response/status manipulation:** Browser-visible response changes must not alter server-side authentication state.
- **Code reuse and expiry:** A code should be single-use and short-lived; confirm repeated submissions fail safely.
- **Backup-code abuse:** Apply rate limits and invalidate each code after use.
- **Previous sessions:** Review whether a session created before MFA activation retains privileges unexpectedly.
- **Referrer-based checks:** Do not rely on a `Referer` header or navigation path as proof that MFA was completed.
- **Response leakage and JavaScript:** Ensure OTPs, recovery secrets, or sensitive verification state are not exposed in responses or frontend bundles.

- **Password-reset interactions:** Define whether password reset changes MFA state; do not silently disable MFA unless the documented recovery policy permits it.
- **Source limitation:** The PDF lists test ideas but does not demonstrate a working bypass against a named application.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Test account: [Controlled-Test-Account]
MFA challenge ID: [Redacted-Test-ID]
Expected: [Protected action denied before successful MFA]
Observed: [Redacted result]
Code reused successfully: [Yes/No]
Unauthenticated/protected access granted: [Yes/No]
Evidence: [Redacted request/response]
```

- [ ] MFA state is verified server-side on every protected route.
- [ ] Challenge, code, account, and session are bound together.
- [ ] OTPs and recovery codes expire and cannot be reused.
- [ ] Retry limits and alerting are enforced.
- [ ] Backup codes are securely generated and stored.
- [ ] MFA reset and disable flows require strong verification.
- [ ] Sensitive data is absent from logs, responses, and frontend bundles.
- [ ] Sessions are rotated or invalidated when authentication state changes.

■■ Mitigation & Remediation

- Maintain explicit server-side authentication states, such as `password_verified` and `mfa_verified`.
- Bind each challenge to the user, session, purpose, and expiry; invalidate it after success or replacement.
- Use cryptographically secure OTP/recovery-code generation and constant-time verification where applicable.
- Rate-limit verification attempts and monitor suspicious patterns.
- Protect MFA management with re-authentication, CSRF defenses, and anti-clickjacking controls.
- Never trust client-controlled cookies, response flags, referrer headers, or frontend logic as proof of MFA completion.
- Define secure session invalidation behavior for password changes, recovery, logout, and MFA changes.

■ Source Notes

Based on the supplied five-page *bypass 2FA.pdf*. Page 1 discusses flawed verification state, clickjacking, response/status handling, code reuse, and CSRF on MFA settings. Page 2 covers backup codes, existing sessions, referrer checks, response leakage, JavaScript review, rate limits, account recovery, and code integrity. The remaining pages contain little or no readable text in the supplied rendering, so no additional claims are inferred.

Educational use only. Stay in scope and disclose findings through the program's approved process.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

403 Forbidden Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ HTTP 403 Forbidden — Access-Control Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Test only in-scope systems. Do not brute-force large path lists, probe unrelated hosts, or attempt to reach administrative data without explicit authorization.*

■ Executive Summary

A `403 Forbidden` response indicates that a server understood a request but refused access. Security testing should determine whether the refusal is consistently enforced across routes, HTTP methods, proxy layers, and identity contexts—not simply whether a response can be changed by altering request metadata.

■ Core Testing Methodology

1. **Confirm scope and baseline.** Record the URL, method, role, status, and expected access policy.
2. **Compare authorized roles.** Use test accounts with different permissions to verify that protected resources remain restricted.
3. **Review route normalization.** In a staging environment, check whether the front end and application interpret paths consistently, including encoded or normalized forms.
4. **Review method handling.** Confirm that GET, POST, and any supported alternate methods enforce the same authorization policy for equivalent actions.
5. **Review proxy/header trust.** Determine whether forwarding or host-related headers are accepted only from trusted infrastructure.
6. **Check canonicalization consistency.** Verify that the reverse proxy, web server, and application agree on the normalized path and resource.
7. **Review discovery controls.** Use a small, approved route list or documented endpoints; do not run uncontrolled brute-force discovery.
8. **Inspect errors safely.** Confirm that denied responses do not leak stack traces, internal paths, or sensitive metadata.
9. **Validate the actual impact.** A changed status code alone is not a vulnerability; confirm whether protected data or an action became available using a test object.
10. **Report and retest.** Include the access policy, test role, request variation category, and redacted evidence.

■ High-Value Bug Bounty Tips & Edge Cases

- **Route discovery:** The source mentions fuzzing and wordlists. Prefer a bounded list of in-scope, likely routes and respect program limits.
- **Header variations:** The PDF lists many proxy and host-related header names. These should never be trusted from arbitrary clients for authorization decisions.
- **Loopback values:** The source shows loopback-style header values. Do not assume such a value is trustworthy; trust must be based on the network boundary and proxy configuration.
- **URL/path variations:** The source contains a large collection of encoded and alternate path forms. Treat these as canonicalization test categories, not a payload list to spray against production.
- **Status-code changes:** A `200` response is not proof of a bypass. Check the response body, session state, and whether protected functionality is actually accessible.
- **Source limitation:** The PDF is primarily a list of tools, header names, and path variations; it does not establish that a particular target is vulnerable.

■■ Educational Checklists & Safe Examples

```
GET /[IN-SCOPE-PROTECTED-RESOURCE] HTTP/1.1
Host: example.com
Authorization: Bearer [CONTROLLED-TEST-TOKEN]
```

Test with an account that is not authorized for the resource.

```
Expected: 403 or another documented safe denial
Observed status: [Status]
Sensitive content returned: [Yes/No]
Action performed: [Yes/No]
Evidence: [Redacted]
```

- [ ] Authorization is enforced by the application, not by obscurity.
- [ ] Proxy headers are accepted only from trusted proxies.
- [ ] Path normalization is consistent across layers.
- [ ] Alternate methods do not bypass access policy.
- [ ] Denied responses do not leak sensitive data.
- [ ] Testing stays within the defined route and request budget.

■■ Mitigation & Remediation

- Enforce authorization at the application/resource layer for every request.
- Configure reverse proxies to strip untrusted forwarding headers and set trusted values themselves.
- Normalize paths once and consistently; reject ambiguous encodings.
- Keep proxy, web server, router, and framework access rules aligned.
- Avoid relying on IP headers, host headers, hidden URLs, or status codes as authorization.
- Log repeated denied access attempts and safely rate-limit abusive discovery.
- Add regression tests for roles, methods, path normalization, and proxy boundaries.

■ Source Notes

Based on the supplied 11-page *bypass 403.pdf*. Page 1 mentions tools, path discovery, and header names; pages 2 and 4 show header-value examples; later pages contain extensive URL/path variants. Because the document is largely a payload catalogue, this rewrite groups the material into safe testing categories rather than reproducing a bulk bypass list.

Educational use only. Test ethically and stay within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Bug Bounty Advance Training Checklist

Educational Bug Bounty Series • Authorized Security Research Only

■ Bug Bounty Learning Checklist — Progress Tracker

Contributed by Cyber_Spy.404 | Bug Bounty Notes

■ Executive Summary

The source HTML, *Bug Bounty Advance Training.html*, is an interactive checklist for tracking learning progress. It groups 55 topics into vulnerabilities, bypass techniques, functionality testing, CVEs, miscellaneous topics, technologies, and reconnaissance. The original UI's counters initialize at zero and are intended to update as boxes are checked.

■ Core Testing Methodology

1. Use this checklist as a learning tracker, not as permission to test a live system.
2. Study the concept and its defensive controls before attempting a lab.
3. Practice in deliberately vulnerable labs or on assets covered by written authorization.
4. Keep notes on prerequisites, test design, expected behavior, evidence, and remediation.
5. Mark a topic complete only when you can explain it and demonstrate it safely.
6. For CVEs, verify the affected product/version and consult vendor advisories rather than assuming every listed year applies.

■ High-Value Bug Bounty Tips & Edge Cases

■ Vulnerabilities (23 items in the HTML)

- [ ] Arbitrary File Upload
- [ ] BugBounty Programs Dorking
- [ ] CRLF Injection
- [ ] Cross Site Request Forgery (CSRF)
- [ ] Cross Site Scripting (XSS)
- [ ] Denial of Service (DoS)
- [ ] Exposed Source Code
- [ ] Fast Automation
- [ ] Host Header Injection
- [ ] Insecure Direct Object References (IDOR)
- [ ] Local File Inclusion (LFI)
- [ ] Mass Assignment
- [ ] NoSQL Injection (NoSQLi)
- [ ] OAuth Misconfiguration
- [ ] Open Redirect
- [ ] Reflected File Download (RFD)
- [ ] Remote File Inclusion (RFI)
- [ ] Server Side Include Injection (SSI Injection)
- [ ] Server Side Request Forgery (SSRF)

- [ ] SQL Injection (SQLi)
- [ ] Web Cache Deception
- [ ] Web Cache Poisoning
- [ ] WordPress Vulnerabilities

■■ Bypass Techniques (4 items in the HTML)

- [ ] Bypass 2FA (Two-Factor Authentication)
- [ ] Bypass 403 Forbidden
- [ ] Bypass 429 Too Many Requests
- [ ] Bypass Captcha

Study bypass categories in labs or under explicit program rules. Do not evade rate limits, access controls, or CAPTCHA on real services without authorization.

■ Functionality Testing (2 items)

- [ ] Forgot Password Functionality
- [ ] Register Functionality

■ CVEs (3 items)

- [ ] CVEs 2021
- [ ] CVEs 2022
- [ ] CVEs 2023

■ Miscellaneous Topics (7 items)

- [ ] Account Takeover
- [ ] Broken Link Hijacking
- [ ] Business Logic Errors
- [ ] Default Credentials
- [ ] Email Spoofing
- [ ] JWT Vulnerabilities
- [ ] Tabnabbing

■ Technologies (12 items)

- [ ] Apache (HTTP Server)
- [ ] Confluence
- [ ] Grafana
- [ ] HAProxy
- [ ] Jenkins
- [ ] Jira
- [ ] Joomla
- [ ] Laravel
- [ ] Moodle
- [ ] Nginx
- [ ] WordPress
- [ ] Zend

■ Reconnaissance (4 items)

- [ ] Scope Based Recon
- [ ] Github Dorks
- [ ] Google Dorks
- [ ] Shodan Dorks

■■ Educational Checklist & Safe Examples

For each topic, record:

```
Topic: [Checklist item]
Learning resource: [Course / Documentation / Lab]
Lab or authorized target: [Name]
Expected behavior: [Summary]
Observed behavior: [Summary]
Defensive fix: [Summary]
Status: [Not Started / Learning / Practiced / Reviewed]
```

- [ ] Explain the vulnerability in your own words
- [ ] Reproduce only in a lab or authorized scope
- [ ] Identify likely impact and affected security boundary
- [ ] Explain the mitigation
- [ ] Save sanitized notes and links to reliable references

■■ Mitigation & Remediation

Use the appropriate secure-development practice for the confirmed issue: strong authorization, input validation, safe output encoding, parameterized database queries, secure file handling, rate limiting, robust authentication, secure token handling, patched dependencies, and regression tests.

■ Source Notes

Based on *Bug Bounty Advance Training.html*. The HTML has 23 vulnerability entries, 4 bypass entries, 2 functionality entries, 3 CVE-year entries, 7 miscellaneous entries, 12 technologies, and 4 reconnaissance entries (55 total). The original interface's displayed “0/22” for vulnerabilities conflicts with its 23 listed items; this Markdown follows the actual item list and calls out the mismatch rather than silently changing the source.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Bug Bounty Research Tips Part 2

Educational Bug Bounty Series • Authorized Security Research Only

■ Bug Bounty Research Tips — Part 2

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Apply reconnaissance and testing techniques only to assets permitted by a bug bounty program. Avoid secret extraction, account takeover, destructive tests, and high-volume scanning.*

■ Executive Summary

The supplied *tips twitter 2.pdf* collects short research notes about XSS filtering, Adobe Experience Manager JSON routes, file-upload extension policy, SQL injection indicators, phpMyAdmin behavior, SVN exposure, and parameter-name injection. The notes are exploratory observations, not proof that the techniques work on every target.

■ Core Testing Methodology

1. Read the program scope and rate limits before reconnaissance.
2. Inventory known, in-scope URLs and identify the technology only when evidence supports it.
3. For AEM, review documented JSON endpoints and expected access behavior without enumerating unrelated content.
4. Review upload policy using harmless files in a controlled environment.
5. Check SQL injection through code review or approved non-destructive test methods.
6. Verify admin/setup interfaces are properly restricted and interpret redirects/status codes in context.
7. Review exposed SVN metadata without downloading or extracting repository contents.
8. Test parameter parsing with harmless markers in a lab.
9. Document reproducible evidence, impact, and uncertainty.

■ High-Value Bug Bounty Tips & Edge Cases

- **XSS filters:** Location concatenation and filtering can produce context-specific behavior. Use a harmless marker and focus on contextual output encoding rather than payload collections.
- **AEM JSON routes:** The source lists alternate `.infinity`/JSON-style routes. Validate only expected public endpoints and do not scrape private content.
- **Upload extensions:** The source labels server-side script extensions as potentially executable and HTML/SVG-like formats as potential XSS. Enforce a strict allowlist and safe serving.
- **SQL injection:** The source describes a sitemap endpoint and time-delay testing. Do not use delay payloads against production unless explicitly permitted; prefer staging or safe validation.
- **phpMyAdmin:** Redirect/status observations alone do not prove setup exposure or authentication bypass. Verify expected access with a controlled environment.
- **SVN:** Exposed metadata can disclose source history; do not use extraction tools on third-party repositories.
- **Parameter-name injection:** Treat this as a validation and query-construction review category.
- **Source limitation:** These are short social-media-style notes, not a validated methodology or universal bypass list.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Scope confirmed: [Yes/No]
Technology evidence: [Observed Indicator]
Test category: [XSS / AEM / Upload / SQLi / Admin Interface / SVN]
Method: [Code Review / Staging / Approved Non-Destructive Test]
Result: [Confirmed / Not Reproduced / Inconclusive]
Evidence: [Redacted]
```

- [ ] Scope and rate limits checked.
- [ ] Only harmless fixtures used.
- [ ] No source repository was extracted.
- [ ] No high-volume or delay-based tests were run without approval.
- [ ] Findings distinguish evidence from hypothesis.

■■ Mitigation & Remediation

Use context-aware output encoding, validate routes and parameters, restrict upload types and execution, parameterize SQL queries, protect setup/admin interfaces, block version-control metadata, and log/review anomalous requests.

■ Source Notes

Based on the supplied three-page *tips twitter 2.pdf*. Page 1 covers XSS filter behavior and AEM JSON route variations. Page 2 lists file extensions, a SQLi/time-delay example, and phpMyAdmin setup behavior. Page 3 includes SVN metadata and parameter-name injection notes. High-risk payloads and repository-dumping instructions are omitted.

Educational use only. Follow program scope and rate limits.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Bug Bounty Research Tips

Educational Bug Bounty Series • Authorized Security Research Only

■ Bug Bounty Research Tips — Field Notes

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Perform reconnaissance only on authorized assets. Never use exposed credentials or retrieve private data to prove a point.*

■ Executive Summary

The supplied five-page *tips twitter.pdf* is a collection of field notes spanning authorization/mass assignment, exposed configuration, cloud/CDN behavior, endpoint discovery, default credentials, leaked tokens, sensitive files, reflected XSS, XXE, SVN, Jira, source maps, AngularJS, and Rails IDOR. Treat each note as a lead to validate carefully, not as a guaranteed exploit.

■ Core Testing Methodology

1. Confirm scope and permitted reconnaissance methods.
2. Inventory in-scope domains and public endpoints using program-approved tools and conservative rates.
3. Review client-side code and source maps for accidentally exposed non-secret implementation details; never publish secrets.
4. Check for public configuration artifacts using minimal requests and stop if sensitive content is encountered.
5. Review authorization using controlled accounts, especially role/tenant/object boundaries.
6. Check authentication and default-credential risk only in a lab or where the program explicitly permits it.
7. Validate XSS, XXE, and injection concerns with harmless fixtures in staging.
8. Review public metadata, source-control remnants, and administrative interfaces without extracting private repositories or using leaked credentials.
9. Use program-approved templates and scanners with low request volume.
10. Record impact, scope, evidence, and limitations; report responsibly.

■ High-Value Bug Bounty Tips & Edge Cases

- **Mass assignment:** The source describes a role field appearing in a response and searching code for alternate role names. The defensive lesson is to keep privileged properties server-controlled and verify authorization independently.
- **Cloud/CDN filtering:** The source includes a CloudFront-related XSS bypass idea. CDN behavior varies; validate with harmless test data and focus on origin-side output encoding.
- **Endpoint discovery:** The source recommends gathering URLs and JavaScript files. Limit collection to in-scope assets and avoid indiscriminate scanning.
- **Configuration exposure:** A response status such as 200/404 is not enough; confirm whether content is actually sensitive without copying it.
- **Authentication:** Default credentials and SQL authentication-bypass notes are unsafe to try on live systems without explicit permission.
- **Leaked tokens:** If a token is exposed, do not use it. Report its location with redacted evidence and let the owner verify/rotate it.
- **Recon and fuzzing:** Tools can generate substantial traffic; honor program rate limits and stop on instability.
- **Jira:** The notes mention default credentials and exposed filters/user lists. Validate access boundaries with controlled accounts.

- **SVN/WC metadata:** Repository metadata may expose source history; do not dump repositories.
- **XSS and XXE:** Use non-destructive validation; never exfiltrate cookies, source code, or local files.
- **Source maps and AngularJS:** These may expose implementation details or indicate legacy client-side behavior, but are not vulnerabilities by themselves.
- **Rails IDOR:** Check record-level authorization with two controlled accounts; URL patterns alone do not prove IDOR.
- **Source limitation:** Several notes are brief and some commands are incomplete. This guide preserves the topic categories without treating their claims as verified findings.

■■ Educational Checklists & Safe Examples

```
Asset: [In-Scope-Asset]
Recon source: [Approved Source]
Observation: [Non-Sensitive Summary]
Security boundary: [Expected Policy]
Test account/fixture: [Controlled Test Data]
Result: [Confirmed / Not Reproduced / Inconclusive]
Sensitive data accessed: [No]
Evidence: [Redacted]
```

- [ ] No out-of-scope assets were queried.
- [ ] No leaked credentials or tokens were used.
- [ ] No private source repositories were extracted.
- [ ] Scanning stayed within program limits.
- [ ] All evidence is redacted and reproducible.

■■ Mitigation & Remediation

Enforce server-side authorization, protect secrets and configuration files, rotate exposed tokens, remove development artifacts, use secure defaults, patch legacy components, apply context-aware output encoding, disable unsafe XML features, protect administrative interfaces, and rate-limit automated requests.

■ Source Notes

Based on the supplied five-page *tips twitter.pdf*. Pages 1–2 cover mass assignment, a CloudFront/XSS note, URL and JavaScript discovery, config exposure, SQL-authentication notes, default credentials, Nmap, API/token tips, SSRF/recon, Swagger, and WC/SVN metadata. Pages 3–4 include asset recon, Jira, source maps, reflected XSS, XXE, cookie-related concerns, private information URLs, exposed development portals, leaked GitHub/AWS/Jira credentials, AngularJS, and Rails IDOR. Page 5 continues with the AngularJS and Rails notes. This rewrite omits credential-use and exfiltration instructions.

Educational use only. Responsible disclosure protects users and systems.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

BYPASS RATE LIMIT Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Rate-Limit Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use only approved test accounts and a program-approved request budget. Do not brute-force credentials, evade protections on third-party services, or generate traffic that could degrade availability.*

■ Executive Summary

Rate limiting reduces automated abuse of login, password-reset, MFA, confirmation-code, and registration workflows. A robust implementation must count requests consistently across relevant routes and identity signals without trusting attacker-controlled headers or allowing trivial session changes to reset the limit.

■ Core Testing Methodology

1. **Identify sensitive endpoints.** Review login, password reset, MFA, confirmation-code, and sign-up flows.
2. **Read the program rules.** Establish the maximum approved request volume and stop conditions before testing.
3. **Establish a baseline.** Send a small number of requests with a controlled account and record response behavior.
4. **Check limit enforcement.** Confirm that the server throttles or blocks requests at the documented threshold.
5. **Review identifier consistency.** Verify that changing a session or cookie does not trivially reset protection where the policy is meant to cover the account or source.
6. **Review proxy configuration.** Ensure client-supplied IP headers cannot arbitrarily change the rate-limit identity.
7. **Review normalization.** In a lab, check whether the same logical identifier is counted consistently when represented with harmless whitespace or encoding variations.
8. **Check concurrent requests carefully.** Use a low-volume controlled test to verify that simultaneous requests cannot exceed a critical one-use limit.
9. **Verify recovery behavior.** Confirm that legitimate users can recover access through documented processes without weakening the control.
10. **Document and retest.** Record the limit, request count, response, and account state without exposing real user data.

■ High-Value Bug Bounty Tips & Edge Cases

- **Where to look:** The source lists login, password reset, 2FA, confirmation codes, and sign-up.
- **Null/encoding edge cases:** The source mentions null-character variants. Treat these as input-normalization tests in a local lab; do not use them to evade a live service's controls.
- **Cookie changes:** Rate limits should not depend solely on a freely replaceable client cookie.
- **Host header:** Host-related values must not be used as a user identity or trusted rate-limit key without a secure design.
- **Forwarding headers:** `X-Forwarded-For` and similar headers should be set by trusted proxies and not accepted arbitrarily from the public internet.
- **Distributed limits:** Consider per-account, per-IP/network, per-device or risk-based controls, with privacy and false-positive trade-offs.
- **Source limitation:** The two-page source is a brief checklist and does not define a particular application's thresholds or demonstrate a working bypass.

■■ Educational Checklists & Safe Examples

```
Endpoint: [In-Scope-Login-or-Recovery-Endpoint]
Test account: [Controlled-Test-Account]
Approved request budget: [Program-Defined-Limit]
```

```
Observed threshold: [Count or Not Determined]
Throttle response: [Redacted]
Changing session resets limit: [Yes/No/Not Tested]
Forwarding headers trusted from clients: [Yes/No/Not Tested]
```

- [ ] Sensitive endpoints have documented limits.
- [ ] Limits are enforced server-side.
- [ ] Account-level limits cannot be reset by changing a cookie.
- [ ] Only trusted proxy infrastructure determines client IP.
- [ ] Concurrent requests cannot bypass one-use or strict limits.
- [ ] Monitoring and user-safe recovery are in place.

■■ Mitigation & Remediation

- Apply layered limits by account, source network, endpoint, and risk signal.
- Trust client IP information only when supplied by configured reverse proxies.
- Normalize identifiers before counting requests and reject malformed values consistently.
- Use atomic counters or equivalent concurrency-safe controls.
- Add progressive delays, alerts, and risk-based challenges where appropriate.
- Avoid revealing exact internal thresholds unnecessarily, while providing usable error and recovery paths.
- Monitor abuse and test limits in staging before deployment.

■ Source Notes

Based on the supplied two-page *BYPASS RATE_LIMIT.pdf*. Page 1 lists sensitive authentication workflows, null-character-related input variations, host-header changes, cookie changes, and forwarding headers. Page 2 shows additional forwarding-header examples. The source does not provide verified results for a particular application.

Educational use only. Stay within scope and the approved request budget.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Command Injection Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ OS Command Injection — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Test only in a local lab or explicitly authorized environment. Do not execute commands that read secrets, alter files, establish persistence, or affect production systems.*

■ Executive Summary

OS command injection occurs when untrusted input is incorporated into an operating-system command and interpreted as command syntax rather than ordinary data. The supplied WSTG-style document focuses on parameters that influence file retrieval and HTTP requests, where unsafe command construction can allow unintended process execution.

■ Core Testing Methodology

1. **Identify candidate inputs.** Review parameters that select files, documents, filenames, or other values likely to reach server-side process calls.
2. **Trace the normal request.** Record the expected request and response for a harmless test file.
3. **Use a controlled test environment.** Reproduce the relevant code path in staging or a local lab before testing any command-injection hypothesis.
4. **Use non-destructive indicators.** Prefer application logs, mocked process calls, or benign fixed test markers rather than commands that access system files or execute arbitrary programs.
5. **Check input handling.** Verify that delimiters, encoded values, and unexpected characters are rejected or treated as literal data.
6. **Review POST and query parameters.** Determine whether different request locations reach the same unsafe process invocation.
7. **Verify server-side impact.** Do not infer command execution from a changed status code alone; use controlled, auditable evidence.
8. **Document the code path.** Record the parameter, expected behavior, observed behavior, and relevant safe evidence.
9. **Retest remediation.** Confirm the vulnerable process call has been removed or made safe.

■ High-Value Bug Bounty Tips & Edge Cases

- **Filename parameters:** File paths and document identifiers are common places to review, but a parameter alone does not prove shell execution.
- **Encoded delimiters:** URL decoding and multiple parsing layers can produce inconsistent validation. Validate the canonical value and avoid repeated decoding.
- **POST body inputs:** The source includes a POST request example involving document retrieval. Check whether body fields reach a shell command.
- **Unsafe concatenation:** Any use of shell command strings with user-controlled input deserves review.
- **Error behavior:** Errors may reveal implementation details but do not alone prove command injection.
- **Source limitation:** The PDF includes legacy examples and does not provide a modern, safe lab or verified result for a specific target.

■■ Educational Checklists & Safe Examples

Example of a safe, non-executable input-validation test:

```
POST /documents/preview HTTP/1.1
Host: example.com
```

```
Content-Type: application/x-www-form-urlencoded

document_id=sample-document-001
```

- [ ] Inputs are treated as data, not shell syntax.
- [ ] The application uses safe APIs rather than shell-string concatenation.
- [ ] File access is constrained to an approved directory.
- [ ] Invalid values fail safely.
- [ ] Logs and errors do not expose secrets or system details.

■■ Mitigation & Remediation

- Avoid invoking a shell where a language-native API can perform the task.
- Use fixed executable paths and argument arrays rather than concatenated command strings.
- Validate identifiers against strict allowlists and map them to server-controlled resources.
- Apply least privilege and isolate processes.
- Restrict file access and execution permissions.
- Log failures safely and add regression tests for malformed and unexpected input.

■ Source Notes

Based on the supplied five-page *Command injection.pdf*, which identifies itself as a WSTG-style “Testing for Command Injection” document. Page 1 explains the issue and shows URL-based file selection examples; page 2 shows a POST-based document retrieval request and a modified-input example. The remaining pages contain little readable text in the supplied preview, so no further source-specific details are inferred.

Educational use only. Never run destructive commands or test outside authorization.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Cookie Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Cookie & Session Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use controlled accounts and synthetic data. Never copy another person's cookies, session IDs, or tokens, and never test destructive payloads against production.*

■ Executive Summary

Cookies often carry session identifiers or influence application state, so weak validation or session handling can lead to data exposure, cross-site scripting, privilege errors, or denial of service. The supplied three-page PDF covers sensitive cookie data, oversized cookies, arbitrary cookie values, parameter pollution, authentication checks, session lifecycle, privilege escalation, and injection risks.

■ Core Testing Methodology

1. **Inventory cookies.** Record names, purpose, domain/path scope, expiry, and security attributes using your own session.
2. **Check sensitive data.** Verify that cookies do not contain passwords, personal data, or unnecessary authorization details.
3. **Review cookie size limits.** Confirm that oversized cookies are rejected safely and cannot cause excessive resource use.
4. **Test trust boundaries.** Determine whether changing a client-side cookie can alter identity, role, organization, or entitlement.
5. **Review duplicate parameters.** Check whether repeated identity fields are handled unambiguously and whether authorization remains enforced.
6. **Review session lifecycle.** Test logout, password reset/change, expiry, and concurrent sessions with controlled accounts.
7. **Review output reflection.** If cookie values are displayed, verify context-appropriate output encoding.
8. **Review injection risks.** Inspect server-side code for unsafe use of cookie values in SQL, template evaluation, or OS commands; do not execute dangerous payloads against live systems.
9. **Check resource limits.** Ensure unusually large cookies and malformed values fail safely.
10. **Document and retest.** Redact cookie values and verify the fix across related endpoints.

■ High-Value Bug Bounty Tips & Edge Cases

- **Sensitive cookie data:** Keep session identifiers opaque and avoid storing unnecessary personal or authorization data in client-readable values.
- **Cookie length:** Enforce sensible size limits at the edge and application layers.
- **Arbitrary cookie injection:** Invalid cookie values should be rejected or ignored safely; they must not cause privilege changes or sensitive stack traces.
- **Mass assignment/parameter pollution:** Duplicate identity parameters can be interpreted inconsistently across layers. Define and enforce one schema.
- **Cookie bomb/DoS:** Excessive cookie sizes can increase parsing and proxy overhead. Test size handling with small, approved lab cases.
- **SQL injection:** Cookie values must be parameterized if used in database queries.

- **Authentication bypass:** Removing a cookie must not accidentally bypass checks; protected endpoints must require valid server-side session state.
- **XSS:** Reflected cookie data must be encoded for the output context.
- **Insufficient session management:** Review expiry, logout invalidation, password-change invalidation, and concurrent-session policy.
- **Privilege escalation:** Roles and organization membership should be derived from trusted server-side records, not editable cookie fields.
- **Session puzzling:** Avoid reusing one session variable for unrelated purposes or trusting ambiguous state.
- **Source limitation:** The PDF contains several dangerous injection examples. This rewrite describes the risk categories without reproducing code-execution payloads.

■■ Educational Checklists & Safe Examples

```
GET /account/preferences HTTP/1.1
Host: example.com
Cookie: session=[YOUR-TEST-SESSION]
```

- [ ] Cookies use `Secure`, `HttpOnly`, and appropriate `SameSite` settings.
- [ ] Session IDs are opaque and unpredictable.
- [ ] Server-side authorization does not rely on editable client fields.
- [ ] Logout and security events invalidate sessions as designed.
- [ ] Cookie size limits are enforced.
- [ ] Duplicate or malformed values are handled consistently.
- [ ] Cookie values are safely encoded and parameterized wherever used.
- [ ] Public evidence redacts session values.

■■ Mitigation & Remediation

- Store authoritative session and authorization state server-side.
- Use unpredictable session IDs and rotate them after authentication or privilege changes.
- Set `Secure`, `HttpOnly`, and suitable `SameSite` cookie attributes.
- Enforce size limits and reject malformed or ambiguous cookies.
- Parameterize database queries and never evaluate cookie data as code.
- Invalidate sessions after logout, password reset, and other risk events.
- Apply output encoding, CSRF protection, and least privilege.
- Avoid logging raw cookies or session tokens.

■ Source Notes

Based on the supplied three-page *cookie.pdf*. Page 1 covers sensitive data, cookie length, arbitrary values, mass assignment, cookie-bomb DoS, SQL injection, parameter pollution, and authentication checks. Page 2 covers XSS, session management, privilege escalation, session puzzling, and unsafe Python/OS command evaluation examples. The examples are not reproduced as operational payloads.

Educational use only. Keep tokens secret and test only accounts you control.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

CRLF Injection Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ CRLF Injection — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Test only in an authorized lab or in-scope application. Use harmless redirect destinations and avoid injecting headers that affect real users.*

■ Executive Summary

CRLF injection occurs when carriage-return and line-feed characters from untrusted input are allowed to alter the structure of an HTTP header or response. The supplied one-page PDF focuses on URL parameters associated with redirects and mentions encoded representations that may be decoded differently by application layers.

■ Core Testing Methodology

1. **Find header-influencing inputs.** Review parameters that affect redirects, language selection, cookies, or response headers.
2. **Record baseline behavior.** Capture the normal request and response for a controlled test value.
3. **Review decoding.** In a lab, check whether the application decodes input once or multiple times and whether validation happens before or after canonicalization.
4. **Check response construction.** Verify that untrusted values cannot create additional response headers or split a response.
5. **Check redirect validation.** Confirm that redirect destinations are validated against an allowlist or constrained to safe relative paths.
6. **Inspect downstream behavior.** Verify that proxies, caches, and application servers handle invalid control characters consistently.
7. **Document evidence.** Record the parameter, response header behavior, and safe impact demonstration.
8. **Retest the fix.** Confirm control characters are rejected or safely encoded.

■ High-Value Bug Bounty Tips & Edge Cases

- **Redirect parameters:** The source recommends looking at parameters that lead to redirects and inspecting 3xx responses.
- **Encoded characters:** Double-encoded and alternate-encoding representations can expose inconsistent decoding. Use inert test values in a lab rather than redirecting users to hostile sites.
- **Response headers:** Inspect `Location` and `Set-Cookie` behavior for unexpected line breaks or additional headers.
- **Multiple layers:** A proxy and application may decode or normalize values differently.
- **Source limitation:** The source is a one-page summary and does not establish a vulnerability in any particular application.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Parameter: [Redirect-or-Header-Related-Parameter]
Test value: [Harmless synthetic marker]
Expected: [Control characters rejected or safely encoded]
Observed response headers: [Redacted]
Unexpected header created: [Yes/No]
Evidence: [Redacted]
```

- [ ] Untrusted input cannot create response headers.
- [ ] Redirect targets are validated.

- [ ] Canonicalization occurs consistently.
- [ ] Invalid control characters are rejected.
- [ ] Proxies and application servers use consistent parsing.

■■ Mitigation & Remediation

- Reject CR and LF characters in values used to construct headers.
- Use framework APIs for response headers rather than manual string concatenation.
- Validate redirect destinations with allowlists or safe relative paths.
- Canonicalize input before validation and avoid repeated decoding.
- Keep reverse proxies, caches, and application servers patched.
- Add regression tests for encoded and malformed control characters.

■ Source Notes

Based on the supplied one-page *CRLF Injection.pdf*. It defines the issue, recommends reviewing redirect-related parameters and 3xx responses, and shows basic, double-encoded, and alternate-encoding examples. This rewrite preserves those concepts without reproducing redirect-to-hostile-site payloads.

Educational use only. Test ethically and stay within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

CSRF Security Testing Guide

Educational Bug Bounty Series • Authorized Security Research Only

■ Cross-Site Request Forgery (CSRF) — Security Testing Guide

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Perform checks only on authorized applications with controlled accounts. Use harmless, reversible test actions and never create a page designed to change another real user's account.*

■ Executive Summary

Cross-Site Request Forgery (CSRF) abuses a browser's existing authenticated state to cause an action the user did not intend. The supplied four-page *CSRF 2.pdf* introduces CSRF, shows GET/POST and JSON-style examples, and discusses multipart requests and token-validation edge cases.

■ Core Testing Methodology

1. **Find state-changing endpoints.** Prioritize profile, email, password, and other sensitive changes.
2. **Record normal behavior.** Capture a request from your own test session and identify its authentication and CSRF controls.
3. **Check methods and content types.** Verify that each supported request format receives the same appropriate protection.
4. **Verify token enforcement.** Check that absent, invalid, expired, or session-mismatched tokens are rejected.
5. **Check token binding.** Confirm tokens are associated with the right user/session and cannot be substituted across test accounts.
6. **Review multipart handling.** Ensure file-upload or multipart forms do not bypass normal CSRF validation.
7. **Check token validation logic.** Presence-only checks and ambiguous token comparisons are insufficient.
8. **Review browser controls.** Confirm cookie `SameSite` settings and origin validation align with the application's needs.
9. **Verify actual state.** A successful HTTP response alone does not prove the action occurred; inspect only your test account.
10. **Document and retest.** Keep evidence redacted and repeat the test after the fix.

■ High-Value Bug Bounty Tips & Edge Cases

- **GET and POST:** Sensitive state changes should not be reachable through a safe method such as GET.
- **JSON requests:** CORS and content-type behavior may affect exploitability, but neither should be treated as the sole CSRF defense.
- **Multipart requests:** Apply CSRF controls to multipart and upload workflows.
- **Token variations:** Check missing, malformed, expired, and session-mismatched tokens in a lab.
- **Duplicate token/cookie patterns:** Validate the intended relationship between the submitted token and cookie; do not accept arbitrary matching values without appropriate integrity.
- **Source limitation:** The source contains proof-of-concept markup and scripts. This rewrite omits deployable cross-site attack code and does not claim a specific application is vulnerable.

■■ Educational Checklists & Safe Examples

```
Endpoint: [In-Scope-State-Changing-Endpoint]
Method/content type: [Documented Format]
CSRF control: [Token / Origin / Framework Middleware]
Test case: [Missing / Invalid / Expired / Cross-Session]
Expected: [Request Rejected]
Observed: [Redacted]
```

```
State changed on test account: [Yes/No]
```

- [ ] CSRF protections cover all state-changing endpoints.
- [ ] Tokens are unpredictable and session-bound.
- [ ] Multipart and alternate content types are covered.
- [ ] Origin checks and cookie policies are configured.
- [ ] GET does not change sensitive state.
- [ ] Invalid requests do not change account state.

■■ Mitigation & Remediation

- Use maintained framework CSRF middleware and correctly validate tokens server-side.
- Apply protection to all state-changing endpoints, including multipart requests.
- Bind tokens to the user session and reject missing, invalid, or expired tokens.
- Validate `Origin` and `Referer` as defense in depth.
- Use `SameSite` cookies where compatible with the application.
- Avoid state changes through GET and test CORS configuration separately.
- Add automated regression tests for token and method edge cases.

■ Source Notes

Based on the supplied four-page *CSRF 2.pdf*. Page 1 defines CSRF and shows GET, POST, and JSON request patterns. Page 2 contains a multipart example and introduces CSRF-token bypass categories; the later pages have limited readable text in the supplied preview. The rewrite preserves the visible subject areas and replaces operational attack code with a safe test record.

Educational use only. Test ethically and stay within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

CSRF Security Testing Methods

Educational Bug Bounty Series • Authorized Security Research Only

■ Cross-Site Request Forgery (CSRF) — Testing Methods

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use a lab or a dedicated test account. Do not trigger state-changing requests for other users or publish live CSRF pages targeting real services.*

■ Executive Summary

CSRF occurs when a browser sends an authenticated request that causes an unwanted state change because the application does not adequately verify that the request was intentionally initiated by the user. The supplied three-page *CSRF 1.pdf* lists missing or weak tokens, referer/content-type checks, request-method variations, token reuse, and token/session-binding concerns.

■ Core Testing Methodology

1. **Identify state-changing actions.** Review email changes, profile updates, password changes, and other operations that use ambient browser credentials.
2. **Inspect defenses.** Check CSRF tokens, `Origin`/`Referer` validation, content-type rules, and cookie `SameSite` settings.
3. **Check token enforcement.** In a controlled test, verify that missing, invalid, or expired tokens are rejected.
4. **Check session binding.** Confirm a token issued to one test session cannot authorize a different session.
5. **Check request-method consistency.** Ensure security checks apply to every supported method and that state-changing actions are not exposed through unsafe GET routes.
6. **Check token lifecycle.** Verify token rotation and invalidation behavior matches the application's design.
7. **Review referer/origin policy.** Confirm validation is robust when headers are absent or malformed; do not rely on a single weak signal.
8. **Review cookie behavior.** Check whether cookies are sent cross-site and whether `SameSite` policy is appropriate.
9. **Record outcomes.** Use a reversible test action and redacted evidence.
10. **Retest remediation.** Add regression tests for all affected actions.

■ High-Value Bug Bounty Tips & Edge Cases

- **Missing or weak tokens:** A token must be unpredictable, validated server-side, and bound to the user session.
- **Content type:** Do not assume a content-type check alone is sufficient protection.
- **Referer/Origin checks:** Validate the expected origin carefully and define behavior for missing headers.
- **Method changes:** State-changing actions should not become available through GET simply because another method is protected.
- **Token presence-only validation:** Presence is not enough; the token must be valid and correctly bound.
- **Token reuse:** Reuse policy should be explicit. A token should not be accepted across unrelated sessions.
- **Method override:** Ensure method-override features cannot bypass CSRF protections.
- **Cookie/header token patterns:** If using a double-submit pattern, bind and validate the token correctly; do not merely check that two arbitrary values exist.
- **Source limitation:** The PDF's examples are proof-of-concept sketches and do not document a specific target's behavior.

■■ Educational Checklists & Safe Examples

```
Action: [Controlled-Test-Action]
Session: [Test-Session-A]
Token state: [Missing / Invalid / Expired / Valid]
Expected: [Invalid requests rejected]
Observed: [Redacted result]
State changed: [Yes/No]
```

- [ ] All state-changing endpoints require CSRF protection where applicable.
- [ ] Tokens are unpredictable and session-bound.
- [ ] Origin validation is correctly configured.
- [ ] GET requests do not perform sensitive state changes.
- [ ] Method overrides cannot bypass protections.
- [ ] Cookies use appropriate `SameSite` settings.
- [ ] Tests use reversible actions and controlled accounts.

■■ Mitigation & Remediation

- Use framework-provided CSRF middleware.
- Validate synchronizer tokens or correctly implemented double-submit tokens.
- Validate `Origin` and, where appropriate, `Referer` as defense in depth.
- Avoid state changes through GET.
- Set cookies with suitable `SameSite`, `Secure`, and `HttpOnly` attributes.
- Ensure all supported content types and method-override paths receive equivalent protection.
- Add regression tests for missing, invalid, expired, and cross-session tokens.

■ Source Notes

Based on the supplied three-page *CSRF 1.pdf*. Page 1 lists general bypass categories and a basic form example; page 2 discusses token validation tied to presence or request method, session binding, method override, and duplicated token/cookie patterns. Page 3 has no additional readable text in the supplied preview. The guide summarizes concepts without reproducing a ready-to-use cross-site attack page.

Educational use only. Stay within scope and disclose findings responsibly.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Denial of Service Security Testing

■ Denial of Service (DoS) — Defensive Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Availability testing can disrupt services. Use staging, local labs, or explicit written authorization with agreed traffic and payload limits. Do not run floods, oversized requests, or resource-exhaustion tests against production without a formal plan.*

■ Executive Summary

Denial of Service (DoS) occurs when requests or inputs consume enough resources to prevent a service from operating normally. The supplied three-page PDF lists several potential resource-exhaustion areas, including oversized cookies and inputs, image processing, regular expressions, compression, HTTP headers, and request-limit behavior.

■ Core Testing Methodology

1. **Agree on a test plan.** Define scope, time window, request budget, monitoring, emergency contact, and stop conditions.
2. **Map resource-intensive features.** Review uploads, image resizing, search, parsing, registration, and endpoints that accept large inputs.
3. **Establish a baseline.** Measure normal latency, error rate, CPU/memory use, and response size in staging.
4. **Test input limits safely.** Use incremental, bounded test values and stop before resource use becomes abnormal.
5. **Review cookie and header limits.** Confirm oversized cookies and headers are rejected with safe client errors.
6. **Review image processing.** Enforce file size, pixel count, dimensions, decoding time, and memory limits.
7. **Review regex and parsing.** Inspect expensive regular expressions and parsers for pathological worst-case behavior.
8. **Review compression and caching.** Check safe decompression limits and ensure cache/proxy layers agree on request interpretation.
9. **Review rate limiting.** Confirm documented limits are effective without attempting a high-volume flood.
10. **Document and retest.** Record metrics, exact safe test bounds, and the observed rejection behavior.

■ High-Value Bug Bounty Tips & Edge Cases

- **Cookie bomb:** Oversized cookies can burden request parsing and proxy infrastructure.
- **Long form values:** Long passwords, email fields, and other inputs should be bounded before expensive processing.
- **Image dimensions:** Large pixel dimensions may consume more memory than compressed file size suggests.
- **Frame/image processing:** Validate content types and resource limits for image transformations.
- **Regex DoS (ReDoS):** Poorly designed regular expressions can have excessive worst-case execution time.
- **Cache-poisoned DoS:** Cache behavior and request normalization can create availability problems; test only with approved, reversible cases.
- **HTTP header oversize:** Proxies and origin servers should enforce compatible limits.
- **Control characters:** Reject malformed header characters consistently.
- **Method override:** Allow only documented override behavior and apply the same authorization and resource controls.
- **Forwarded headers:** Trust only configured proxies and avoid letting arbitrary header values alter routing or cache behavior.
- **Source limitation:** The source includes payload-like examples and links, but does not establish that every technique works against a given service.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain-or-Staging]
Feature: [Upload / Image / Form / Header / Cookie / Parser]
Approved maximum input: [Test-Plan-Limit]
Baseline latency: [Value]
Test latency/error rate: [Value]
Resource anomaly: [Yes/No]
Stop condition reached: [Yes/No]
Evidence: [Metrics and redacted logs]
```

- [ ] Request body, header, cookie, and field sizes are bounded.
- [ ] Uploads are checked for type, size, dimensions, and processing cost.
- [ ] Expensive operations have timeouts and concurrency limits.
- [ ] Regex and parsers have bounded worst-case behavior.
- [ ] Compression/decompression limits are enforced.
- [ ] Proxy and origin limits are compatible.
- [ ] Rate limits and monitoring are enabled.
- [ ] Stop conditions are defined before testing.

■■ Mitigation & Remediation

- Enforce limits for request bodies, headers, cookies, fields, uploads, image dimensions, and parser depth.
- Apply request timeouts, concurrency caps, queues, and resource budgets.
- Replace vulnerable regular expressions and use safe parsing libraries.
- Limit decompression ratios and reject malformed compressed input.
- Normalize proxy/cache behavior and keep infrastructure components patched.
- Rate-limit sensitive and expensive endpoints and monitor abnormal resource use.
- Return safe errors without leaking internal details.
- Run load and resilience tests in staging with a written plan and clear stop conditions.

■ Source Notes

Based on the supplied three-page *DOS.pdf*. Page 1 covers cookie bombs, long form inputs, image/frame processing, parameter changes, rate-limit behavior, ReDoS, cache-related DoS, and oversized HTTP headers. Page 2 discusses oversized headers, meta/control characters, method-override headers, and forwarding headers. The final page has limited readable text in the supplied preview. This rewrite converts the source into a controlled defensive testing guide and omits destructive payloads.

Educational use only. Availability testing requires explicit authorization and careful limits.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Django Security Review

Educational Bug Bounty Series • Authorized Security Research Only

■ Django Security Review — Debug Exposure & Endpoint Discovery

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Review only authorized deployments. Do not enumerate large path lists or access debug pages containing secrets on production systems.*

■ Executive Summary

Django applications can expose sensitive implementation details when debug features are enabled or when operational endpoints are unintentionally public. The supplied one-page PDF references Django debug-mode exposure, a debug panel, a wordlist, and example API/health-check paths.

■ Core Testing Methodology

1. **Identify the framework.** Use passive evidence and authorized documentation to determine whether the application uses Django.
2. **Check debug configuration.** Verify in deployment configuration that debug mode is disabled in production.
3. **Review error handling.** Trigger only benign, expected errors in a staging environment and confirm responses do not expose tracebacks, environment variables, or secrets.
4. **Review operational endpoints.** Inventory documented health checks, API routes, and administrative endpoints.
5. **Check access control.** Ensure internal dashboards and sensitive API routes require authentication and appropriate roles.
6. **Use bounded discovery.** If route discovery is allowed, use a small, approved list and respect rate limits.
7. **Check deployment separation.** Ensure development tools and test endpoints are not included in production builds.
8. **Document and retest.** Record configuration evidence and sanitized responses.

■ High-Value Bug Bounty Tips & Edge Cases

- **Debug mode:** Production debug pages can expose source paths, settings, and request metadata.
- **Debug panels:** Developer tooling should not be reachable by untrusted users in production.
- **Health checks:** Health endpoints should return only the minimum operational information needed.
- **API route inventory:** The source lists API, reporting, transaction, organization, and document-like route patterns. Treat these as examples for reviewing your own route inventory, not a target-specific wordlist.
- **Source limitation:** The PDF is a short reference list and does not demonstrate a vulnerability in a particular Django application.

■■ Educational Checklists & Safe Examples

```
Application: [Your-Domain]
Framework: [Confirmed / Suspected Django]
Production DEBUG: [False / Unknown]
Debug details exposed: [Yes/No]
Sensitive route: [Redacted]
Authentication required: [Yes/No]
Evidence: [Redacted]
```

- [ ] `DEBUG` is disabled in production.
- [ ] `ALLOWED_HOSTS` is configured correctly.
- [ ] Secrets are stored outside source control.

- [ ] Error pages do not expose settings or tracebacks.
- [ ] Health checks reveal minimal information.
- [ ] Admin and API routes enforce authorization.
- [ ] Development tools are excluded from production.

■■ Mitigation & Remediation

- Set `DEBUG = False` in production and configure safe error pages.
- Configure `ALLOWED_HOSTS` narrowly and use HTTPS.
- Keep secrets in a managed environment or secret store, not source control.
- Restrict admin and operational endpoints with authentication, network controls, and least privilege.
- Minimize health-check output and protect diagnostic tools.
- Maintain dependencies and add deployment checks that fail if unsafe debug settings are enabled.

■ Source Notes

Based on the supplied one-page *Django.pdf*. It references remote-code-execution/debug-mode reading, a Django debug-panel report, a wordlist, and route examples including API, health-check, transaction, reporting, organization, and document paths. This rewrite treats those items as review prompts and does not reproduce external exploit instructions.

Educational use only. Test ethically and stay within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

EXIF Metadata Privacy Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Image EXIF Metadata Exposure — Privacy Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use images you own or synthetic samples. Never publish a person's precise location or private device details.*

■ Executive Summary

Images may contain EXIF metadata such as GPS coordinates, device make/model, software details, and timestamps. The supplied one-page PDF describes a privacy risk when uploaded images are served without stripping sensitive metadata.

■ Core Testing Methodology

1. Identify an authorized image-upload feature.
2. Use an owned or synthetic image with known, non-sensitive metadata.
3. Upload through the normal workflow with a test account.
4. Retrieve only your own uploaded image.
5. Inspect metadata locally with a trusted EXIF viewer.
6. Compare the source image with the uploaded/downloaded version.
7. Redact coordinates, serial numbers, and private device details from evidence.
8. Report and retest after remediation.

■ High-Value Bug Bounty Tips & Edge Cases

• GPS metadata may reveal latitude and longitude.

• Device/software fields can disclose details about the uploader's environment.

• Thumbnail generation may strip some fields but leave others.

• The original file may retain metadata even when a preview is sanitized.

• Metadata exposure and unauthorized image access are separate issues.

• The source outlines sample-image testing but does not establish a flaw in a specific service.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Test account: [Controlled-Test-Account]
Image: [Synthetic-or-Owned-Image]
Metadata before upload: [Known test fields]
Metadata after download: [Present / Removed / Partial]
Sensitive values: [Redacted]
```

• [ ] GPS and unnecessary EXIF fields are removed where appropriate.

• [ ] Thumbnails and originals follow the same privacy policy.

• [ ] Original uploads are access-controlled.

• [ ] Reports redact all sensitive metadata.

■■ Mitigation & Remediation

Strip GPS and unnecessary EXIF/XMP metadata during upload or processing, re-encode images using maintained libraries, apply the policy to thumbnails and originals, protect original files, and add automated tests using synthetic images.

Source Notes

Based on the supplied one-page *meta data not extracted.pdf*. It describes checking an uploaded image URL with an EXIF viewer and references sample images and a metadata-inspection site.

Educational use only. Protect personal data.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Exposed Source Code Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Exposed Source Code — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Test only authorized systems. Do not dump or publish third-party repositories, secrets, or personal data.*

■ Executive Summary

Private source code and version-control metadata can become publicly accessible through a misconfigured web server. The supplied PDF covers Git, Subversion (SVN), Mercurial (Hg), and Bazaar (Bzr) metadata directories.

■ Core Testing Methodology

1. Confirm the host and paths are in scope.
2. Check, using minimal requests, whether version-control metadata is publicly served.
3. Determine whether directory listing or metadata access is enabled.
4. Do not clone or dump a third-party repository; collect only minimal, redacted evidence.
5. Explain potential exposure of source, internal paths, history, or accidentally committed secrets.
6. Report responsibly and retest after remediation.

■ High-Value Bug Bounty Tips & Edge Cases

• The source lists `/.git`, `/.svn`, `/.hg`, and `/.bzr` as metadata exposure examples.

• A directory listing can disclose repository structure.

• Secrets removed from current code may remain in version history; rotate any confirmed exposed credentials.

• The source links to repository extraction tools, but this guide does not reproduce dumping instructions.

• These examples do not establish that a particular target is vulnerable.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Metadata category: [Git / SVN / Hg / Bzr]
Publicly accessible: [Yes / No / Inconclusive]
Directory listing enabled: [Yes / No]
Evidence: [Minimal redacted status/response]
```

• [ ] Development metadata is excluded from deployments.

• [ ] Directory indexing is disabled where unnecessary.

• [ ] Secrets are scanned and kept outside source control.

• [ ] Confirmed exposed credentials are rotated.

■■ Mitigation & Remediation

Exclude repository metadata from public web roots, deny access to version-control paths, deploy clean build artifacts, disable unnecessary directory listing, scan repositories for secrets, and rotate any exposed credentials.

■ Source Notes

Based on the supplied three-page *Exposed Source Code.pdf*. Pages 1–2 discuss Git, SVN, Mercurial, and Bazaar metadata directories and tool references. The last page has no additional readable text in the supplied preview.

Educational use only. Stay within scope and disclose findings responsibly.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

File Upload Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ File Upload — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use a local lab or authorized staging environment. Do not upload executable payloads to production systems.*

■ Executive Summary

File-upload flaws occur when an application trusts client-controlled filenames, extensions, MIME types, or file content. The supplied PDF covers MIME changes, extension variations, filename edge cases, size validation, and risks associated with different formats.

■ Core Testing Methodology

1. Map upload features, permitted formats, size limits, and storage behavior.
2. Upload a harmless file of an allowed type using a test account.
3. Verify server-side content validation rather than trusting the client `Content-Type`.
4. Check filename normalization and ensure paths cannot be changed or unrelated files overwritten.
5. Verify size, image-dimension, and processing-time limits.
6. Confirm uploads cannot execute as server-side code and private files require authorization.
7. Use harmless malformed-file and MIME-mismatch fixtures in staging.
8. Record redacted evidence and retest the fix.

■ High-Value Bug Bounty Tips & Edge Cases

- The PDF shows changing multipart content types; these values are client-controlled.
- It lists double extensions, uncommon extensions, mixed capitalization, null-byte and path-like filename ideas. Treat these as legacy test categories, not guaranteed bypasses.
- Content-length controls should be enforced at proxy and application layers.
- Images, SVG, HTML, XML, CSV, and document formats have different processing risks.
- Neither an extension allowlist nor a blocklist replaces content validation and safe storage.
- The source includes executable upload examples; they are intentionally omitted here.

■■ Educational Checklists & Safe Examples

```
POST /uploads HTTP/1.1
Host: example.com
Content-Type: multipart/form-data; boundary=[BOUNDARY]

--[BOUNDARY]
Content-Disposition: form-data; name="file"; filename="sample.txt"
Content-Type: text/plain

Harmless test document.
--[BOUNDARY]--
```

- [ ] Actual file content is validated.
- [ ] Upload size and processing limits are enforced.
- [ ] Server-generated filenames are used.
- [ ] Upload directories do not execute scripts.

- [ ] Private uploads require authorization.
- [ ] Active content is sanitized or served from an isolated origin.

■■ Mitigation & Remediation

Use a strict allowlist, validate signatures and parse content with maintained libraries, generate safe filenames, store uploads outside executable web roots, impose size and processing limits, re-encode images when appropriate, isolate active content, and enforce authorization on downloads.

■ Source Notes

Based on the supplied four-page *FILE UPLOAD.pdf*. Pages 1–3 cover content-type and extension variations, size checks, filename cases, format risks, and allow/blocklist concepts. Dangerous executable payloads are not reproduced.

Educational use only. Test uploads in a controlled environment.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

GitHub Dorking Safe Guide

Educational Bug Bounty Series • Authorized Security Research Only

■ GitHub Dorking — Authorized Repository Auditing

Contributed by Cyber_Spy.404 | Bug Bounty Notes

Authorization rule: *Search repositories you own or are authorized to audit. If you discover a real secret in someone else's repository, do not use it, validate it against a service, or copy it into your notes.*

■ Executive Summary

The source HTML, *GitHub Dorking.html*, contains a very large list of search terms associated with API keys, cloud credentials, tokens, passwords, private keys, service accounts, deployment settings, and other configuration indicators. This guide keeps the source's purpose—finding accidental exposure during reconnaissance—but does not reproduce the full operational secret-search list or its many credential-shaped patterns.

■ Core Testing Methodology

1. Confirm the organization, repository, and account are in scope.
2. Prefer GitHub secret scanning, push protection, dependency/security alerts, and organization audit features.
3. Search your own repositories for configuration files, test fixtures, committed environment files, and secret-like placeholders.
4. Review commit history and forks within your authorized environment; deleted files can remain in history.
5. Never test a discovered token against a live API. Treat it as compromised and rotate/revoke it through the owner.
6. Store evidence as a redacted path, commit hash, and short context—not as the credential value.
7. Add prevention controls and verify that the secret is no longer active.

■ High-Value Tips & Edge Cases

- Search terms can identify likely locations but produce false positives (test fixtures, examples, documentation, and revoked credentials).
- A secret may be valid even if it looks unusual; do not validate it by using it.
- Removing a file from the latest commit does not remove it from Git history, forks, caches, or clones.
- Public client identifiers are not always secrets. Assess what privilege the value grants and follow vendor guidance.
- Search syntax and indexing change over time; confirm current behavior in GitHub's official documentation.
- The original file contains thousands of keyword/pattern entries. To keep this guide safe and maintainable, it is summarized by category rather than copied line by line.

■■ Educational Checklists & Safe Examples

Examples below are for **your own organization/repositories only**. Use a placeholder organization name.

```
org:YOUR-OWN-ORG filename:.env
org:YOUR-OWN-ORG filename:config
org:YOUR-OWN-ORG path:.github
org:YOUR-OWN-ORG "EXAMPLE_PLACEHOLDER_VALUE"
repo:YOUR-OWN-ORG/YOUR-OWN-REPO "TODO: rotate test credential"
```

These examples are intended to find configuration hygiene issues and placeholders, not to extract or use real credentials.

- [ ] Scope verified
- [ ] Secret-scanning alerts reviewed
- [ ] Environment/configuration files checked

- [ ] Commit history and forks considered
- [ ] Potential secrets redacted from evidence
- [ ] Exposed values revoked/rotated by the owner
- [ ] Push protection or pre-commit scanning enabled

■■ Mitigation & Remediation

Use a secrets manager or environment injection, never commit live secrets, enable secret scanning and push protection, apply least privilege and short token lifetimes, rotate exposed credentials immediately, and consider history rewriting only as a coordinated cleanup after revocation. History rewriting does not replace revocation.

■ Source Notes

Based on *GitHub Dorking.html*. The source includes many keyword families such as cloud-provider keys, API credentials, chat/service tokens, database passwords, private-key headers, CI/CD secrets, and environment variables. This document preserves the categories and workflow but omits the raw credential-finding catalogue and credential-like strings.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Google Dorking Safe Guide

Educational Bug Bounty Series • Authorized Security Research Only

■ Google Dorking — Authorized Bug Bounty Reconnaissance

Contributed by Cyber_Spy.404 | Bug Bounty Notes

Authorization rule: *Search only for assets and information you are permitted to assess. Do not use search results to access leaked credentials, private files, or personal data.*

■ Executive Summary

The source HTML, *google dorks.html*, presents Google search operators for narrowing results by domain, URL, page title, text, file extension, and third-party code/documentation platforms. Several original examples target exposed secrets and sensitive files; this guide preserves the search-operator concepts but replaces credential-hunting examples with safer, scope-bound templates.

■ Core Testing Methodology

1. Read the bounty scope and substitute only the exact authorized domain.
2. Start with low-impact discovery of public pages, documentation, and known routes.
3. Use operators to narrow results; search results are not proof of a vulnerability.
4. Do not open, download, or copy content that appears to contain secrets, personal data, or private records.
5. Report accidental exposure with a URL and redacted screenshot/snippet; let the owner validate and rotate secrets.
6. Recheck findings after remediation and avoid repeated automated queries that create unnecessary load.

■ High-Value Tips & Edge Cases

- `site:` limits results to a domain or site.
- `inurl:` looks for text in a URL.
- `intitle:` looks for text in a page title.
- `intext:` searches page text.
- `filetype:` or `ext:` narrows by file type.
- Quoted phrases can help locate exact public wording, but indexing can be stale.
- A result may be cached, removed, or inaccessible; do not infer exposure from a snippet alone.
- The source includes queries for private keys, passwords, environment files, and database exports. Those exact secret-hunting queries are intentionally not reproduced here.

■■ Educational Checklists & Safe Examples

Replace `YOUR-AUTHORIZED-DOMAIN.example` with an asset explicitly listed in the program scope.

```
site:YOUR-AUTHORIZED-DOMAIN.example
site:YOUR-AUTHORIZED-DOMAIN.example inurl:docs
site:YOUR-AUTHORIZED-DOMAIN.example intitle:"API documentation"
site:YOUR-AUTHORIZED-DOMAIN.example inurl:status
site:YOUR-AUTHORIZED-DOMAIN.example filetype:pdf
site:YOUR-AUTHORIZED-DOMAIN.example "security.txt"
```

For research on public documentation or your own projects, the source also lists third-party sites such as npm, GitLab, GitHub, Bitbucket, Trello, CodePen, Pastebin, and Prezi. Restrict queries to material relevant to your authorized asset and avoid collecting secrets.

- [ ] Domain is explicitly in scope
- [ ] Search purpose is documented

- [ ] Only public, non-sensitive results are reviewed
- [ ] No leaked credentials are used
- [ ] Evidence is minimal and redacted
- [ ] Search result freshness is checked before reporting

■■ Mitigation & Remediation

Avoid publishing backups, logs, database dumps, private configuration, or secrets in web roots and public repositories. Use access controls, deployment exclusions, secret scanning, and prompt credential rotation when exposure is confirmed. Request removal of stale indexed material where appropriate, but fix the origin first.

■ Source Notes

Based on *google dorks.html*. The source lists domain/URL/title/text/file-extension operators and queries for public code/documentation platforms. It also includes queries targeting passwords, private keys, API secrets, logs, and database files; those operational secret-finding strings are replaced with safe, scope-bound examples.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Host Header Injection Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ HTTP Host Header Injection — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use authorized applications, a test domain you control, and a controlled inbox. Never send misleading links to real users.*

■ Executive Summary

Host header injection occurs when an application trusts unvalidated `Host` or forwarding headers while constructing links, redirects, or security decisions. The supplied PDF highlights password-reset/newsletter workflows, duplicate Host headers, override headers, and absolute URLs.

■ Core Testing Methodology

1. Identify URL-generating features such as password reset, invitations, newsletters, and redirects.
2. Record normal behavior using a controlled test account.
3. Verify only configured canonical hostnames are accepted.
4. Check whether forwarding headers are set only by trusted proxies.
5. Ensure duplicate Host values and ambiguous absolute-form requests are rejected.
6. Inspect generated links using a test inbox.
7. Capture minimal, redacted evidence and retest remediation.

■ High-Value Bug Bounty Tips & Edge Cases

- Password-reset and newsletter features are highlighted by the source.
- The PDF lists `X-Forwarded-For`, `X-Forwarded-Host`, `X-Client-IP`, `X-Remote-IP`, `X-Remote-Addr`, and `X-Host`.
- Forwarding headers must not be trusted when supplied directly by arbitrary clients.
- Proxy and application layers may interpret duplicate Host headers differently.
- An unexpected host value alone is not proof of impact; verify generated behavior safely.
- The source does not establish that a specific target is vulnerable.

■■ Educational Checklists & Safe Examples

```
GET /account/recovery HTTP/1.1
Host: [Your-Approved-Test-Domain]

Expected origin: https://[Configured-Canonical-Domain]
Unexpected host accepted: [Yes/No]
Evidence: [Redacted]
```

- [ ] Valid hostnames are allowlisted.
- [ ] Untrusted forwarding headers are stripped or ignored.
- [ ] Duplicate Host values are rejected.
- [ ] Email links use a configured canonical origin.
- [ ] Cache and routing behavior is consistent.

■■ Mitigation & Remediation

Use an explicit hostname allowlist, configure trusted proxies to overwrite forwarding headers, reject duplicate or malformed Host values, construct absolute links from a configured canonical origin, and test cache/routing behavior.

Source Notes

Based on the supplied two-page *HOST HEADER INJECTION.pdf*. Page 1 describes Host manipulation and duplicate headers; page 2 lists forwarding headers and absolute-URL handling and references PortSwigger.

Educational use only. Stay in scope and use controlled inboxes.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

IDOR Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Insecure Direct Object Reference (IDOR) — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use authorized targets, controlled test accounts, and synthetic records.*

■ Executive Summary

IDOR is an access-control flaw where an application exposes an object reference but fails to verify the caller's permission to use it. The supplied six-page *IDOR.pdf* introduces API identifiers, parameter pollution, JSON formats, and API-version differences.

■ Core Testing Methodology

1. Identify object identifiers in API paths, query strings, and request bodies.
2. Create synthetic objects owned by two controlled accounts.
3. Establish the expected access policy for each account.
4. Review extra object-reference fields and alternate request formats.
5. Check duplicate parameters and JSON structure handling in a lab.
6. Verify legacy routes and methods enforce the same authorization rules.
7. Review read and write actions separately.
8. Record sanitized evidence without accessing third-party data.

■ High-Value Bug Bounty Tips & Edge Cases

• Extra object-ID parameters can reveal weak authorization if the server trusts client selection.

• Duplicate keys can be interpreted inconsistently.

• Arrays and nested JSON objects should be schema-validated.

• Legacy API versions may enforce weaker controls.

• Numeric, UUID, encoded, or hashed IDs still require authorization.

• Publicly indexed URLs do not authorize access to private objects.

• Only the first two pages are clearly readable in the supplied preview; no claims are inferred from unseen content.

■■ Educational Checklists & Safe Examples

```
GET /api/v1/records/[OWN-TEST-RECORD-ID] HTTP/1.1
Host: example.com
Authorization: Bearer [CONTROLLED-TEST-TOKEN]
```

• [ ] Ownership is checked for every read/write.

• [ ] Malformed and duplicate parameters are handled safely.

• [ ] API versions and routes share consistent controls.

• [ ] IDs are never treated as permission grants.

■■ Mitigation & Remediation

Implement object-level authorization on every endpoint, scope data queries to the authenticated user, validate request schemas, reject ambiguous duplicate keys, and test access control across users, tenants, methods, and versions.

Source Notes

Based on the supplied six-page *IDOR.pdf*. Page 1 introduces IDOR and object parameters; page 2 covers JSON suffixes, older API versions, and array-form identifiers. The remaining pages are not sufficiently readable in the supplied preview to support further source-specific claims.

Educational use only. Disclose findings responsibly.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

IDOR Testing Advanced

Educational Bug Bounty Series • Authorized Security Research Only

■ IDOR & Broken Object-Level Authorization — Advanced Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use two test accounts you control and synthetic records. Never access or modify another real user's data.*

■ Executive Summary

IDOR/BOLA occurs when an application fails to verify that the current user may access a requested object. The supplied *IDOR 2.pdf* lists methods, parameter names, paths, content types, identifier formats, API versions, and GraphQL as testing areas.

■ Core Testing Methodology

1. Create two controlled accounts and harmless records owned by each.
2. Identify object IDs in paths, query strings, request bodies, and GraphQL operations.
3. Confirm each account can access its own records but not the other's.
4. Review read, update, delete, export, and sharing operations using synthetic records only.
5. Check that all supported methods and content types enforce authorization.
6. Review duplicate parameters, alternate path forms, and structured JSON values in a lab.
7. Check documented legacy API versions and GraphQL nested resolvers.
8. Document the permission rule, controlled result, and redacted evidence.

■ High-Value Bug Bounty Tips & Edge Cases

- Predictable IDs are not required for IDOR; opaque or encoded IDs are not authorization controls.
- Duplicate parameters may be parsed differently by gateways and applications.
- Method changes and content-type changes must not bypass authorization.
- Legacy API versions may have inconsistent controls.
- GraphQL must enforce permissions in nested resolvers and batch operations.
- Wildcard and bulk endpoints must filter each returned object.
- The PDF contains many bypass-shaped examples; this guide groups them into safe test categories rather than reproducing a bulk list.

■■ Educational Checklists & Safe Examples

```
GET /api/v1/records/[OWN-TEST-RECORD-ID] HTTP/1.1
Host: example.com
Authorization: Bearer [TEST-ACCOUNT-A-TOKEN]
```

- [ ] Every object action checks ownership and permissions.
- [ ] Duplicate parameters are rejected or parsed consistently.
- [ ] All API versions and methods enforce the same policy.
- [ ] GraphQL nested and batch operations enforce authorization.
- [ ] Tests use only synthetic records.

■■ Mitigation & Remediation

Enforce server-side object-level authorization on every action, scope database queries to permitted objects, reject ambiguous parameters, apply consistent controls across API versions and methods, and add regression tests using separate owners.

Source Notes

Based on the supplied two-page *IDOR 2.pdf*. Page 1 lists baseline checks, method and parameter variations, path handling, content types, ID formats, and role-level routes. Page 2 covers indexed endpoints, hidden parameters, parameter pollution, JSON structures, API versions, and GraphQL.

Educational use only. Test only records you control.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Jira Security Review

■ Jira Security Review — Authorized Configuration & Vulnerability Checks

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Assess only Jira instances explicitly in scope. Do not execute exploit payloads or access private project data.*

■ Executive Summary

The supplied four-page *jire.pdf* is a reference list of Jira-related vulnerability reports and CVE identifiers. It mentions SSRF, XSS, information disclosure, user enumeration, path traversal, template injection, project-key enumeration, and authorization issues.

■ Core Testing Methodology

1. Confirm the authorized host, deployment type, and version.
2. Compare version and enabled plugins with vendor security advisories.
3. Use controlled accounts to review anonymous and authenticated permissions.
4. Check that permission APIs reveal only appropriate information.
5. Review project/user disclosure according to the instance's intended privacy settings.
6. Inventory plugins and check their routes and patch status.
7. Validate high-risk input handling only in staging with benign tests.
8. Document the affected component, version evidence, and sanitized result.

■ High-Value Bug Bounty Tips & Edge Cases

- The source lists CVEs from different years and components; verify each against vendor advisories before asserting impact.
- Plugin endpoints may have distinct authentication behavior.
- User-picker, project metadata, error messages, and configuration resources may disclose information if access controls are weak.
- SSRF and template-injection testing should use non-destructive lab methods; do not access internal services or execute commands.
- A permission response must be interpreted against expected instance policy.
- The PDF is a collection of references, not a complete reproduction guide.

■■ Educational Checklists & Safe Examples

```
Instance: [Authorized-Jira-Host]
Version: [Version]
Plugin/component: [Name]
Advisory/CVE: [Verified-Identifier]
Test role: [Anonymous / Test User / Test Admin]
Expected access: [Policy]
Observed behavior: [Redacted]
```

- [ ] Version and plugin inventory is current.
- [ ] Vendor advisories are checked.
- [ ] Anonymous/user permissions match policy.

- [ ] Plugins are patched and unnecessary plugins disabled.
- [ ] No command execution or internal-service access is attempted.

■■ Mitigation & Remediation

Upgrade to fixed vendor-supported versions, patch plugins, disable unused components, restrict administrative access, apply least-privilege project permissions, minimize API disclosure, and regression-test authorization.

■ Source Notes

Based on the supplied four-page *jire.pdf*. Pages 1–2 list Jira CVE references and topics including SSRF, XSS, information disclosure, path traversal, template injection, permissions endpoints, and authorization. Pages 3–4 have limited readable text in the preview.

Educational use only. Verify advisories and remain within scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

JSON Authentication Endpoint Testing

■ JSON Authentication Endpoint Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use controlled accounts and approved request limits. Do not brute-force credentials or target other users.*

■ Executive Summary

JSON authentication endpoints must validate input types and structure consistently, reject malformed or ambiguous data, and apply the same authentication controls as other login paths. The supplied 17-page *json.pdf* catalogs variations involving nulls, data types, malformed JSON, duplicate keys, encoding, nested objects, and injection-like strings.

■ Core Testing Methodology

1. Confirm the endpoint and test scope.
2. Capture a valid baseline request with a controlled account.
3. Check missing, empty, and null required fields.
4. Check strings, numbers, booleans, arrays, and objects against the documented schema.
5. Check malformed JSON and invalid escapes.
6. Verify duplicate keys are rejected or handled consistently.
7. Check length, Unicode, and nesting limits.
8. Ensure errors do not reveal credentials or stack traces.
9. Use harmless fixtures or code review to validate injection defenses.
10. Verify rate limits within the approved budget and document results.

■ High-Value Bug Bounty Tips & Edge Cases

- Empty/null values must not be treated as valid credentials accidentally.
- Type coercion can cause inconsistent validation across layers.
- Reject unsupported nested structures and ambiguous duplicate keys.
- Normalize Unicode consistently and validate canonical values.
- Limit long inputs and nested structures.
- Unexpected fields must not allow clients to set roles or privileges.
- The source is a dense catalogue; this guide consolidates visible categories rather than claiming every example was exhaustively transcribed.

■■ Educational Checklists & Safe Examples

```
POST /api/auth/login HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "username": "[CONTROLLED-TEST-USERNAME]",
  "password": "[TEST-PASSWORD]"
}
```

- [ ] Required fields and data types are enforced.
- [ ] Duplicate keys and malformed JSON are handled safely.

- [ ] Input lengths and nesting are bounded.
- [ ] Errors do not reveal credentials or internals.
- [ ] Rate limits apply consistently.
- [ ] Privilege fields cannot be set through login input.

■■ Mitigation & Remediation

Validate requests against an explicit schema, reject malformed or ambiguous fields, bound lengths and nesting, use parameterized database operations, return generic authentication errors, avoid logging secrets, enforce rate limits, and keep roles server-controlled.

■ Source Notes

Based on the supplied 17-page *json.pdf*, titled as JSON tests for authentication endpoints. Visible pages list empty/null, numeric, boolean, array/object, special-character, malformed, missing/extra/repeated-key, Unicode, escape, whitespace, long-value, and control-character categories.

Educational use only. Respect scope and request limits.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Mass Assignment Security Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Mass Assignment — Security Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Use a test account in an authorized environment. Never attempt to grant yourself real administrative privileges.*

■ Executive Summary

Mass assignment occurs when client input is automatically mapped to internal model properties without an explicit allowlist. The supplied three-page *mass.pdf* focuses on registration and account workflows where role, administrator, or organization-access fields might be accepted unintentionally.

■ Core Testing Methodology

1. Identify registration, profile, password-reset, email-change, and organization endpoints.
2. Record the documented fields for a normal request.
3. Establish a baseline with a synthetic test account.
4. In staging, submit a harmless unknown field and verify it is rejected or ignored.
5. Confirm role, permission, ownership, and verification fields are assigned only server-side.
6. Check field naming/casing and nested-object handling.
7. Verify sensitive actions still require independent authorization.
8. Record sanitized evidence and retest.

■ High-Value Bug Bounty Tips & Edge Cases

• The source shows registration examples with added role/admin-like fields.

• Alternate capitalization or field aliases can reveal inconsistent model binding.

• Organization membership should use a verified invitation or administrative workflow.

• Sensitive email/password changes require the intended identity verification.

• Unknown fields should be rejected or ignored consistently.

• The source does not establish a vulnerability in a particular application.

■■ Educational Checklists & Safe Examples

```
POST /api/v1/register HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "username": "[TEST-USERNAME]",
  "email": "[TEST-EMAIL]",
  "password": "[TEST-PASSWORD]"
}
```

• [ ] Endpoints allowlist writable fields.

• [ ] Roles/admin/ownership fields remain server-controlled.

• [ ] Nested objects are validated recursively.

• [ ] Unknown fields are handled consistently.

• [ ] Authorization is independent of input validation.

■■ Mitigation & Remediation

Use explicit DTOs/serializers and allowlisted fields, separate public profile models from internal authorization models, assign roles only through trusted workflows, validate nested data, and regression-test unexpected privileged fields.

■ Source Notes

Based on the supplied three-page *mass.pdf*. Pages 1–2 cover account registration, organization access, password reset, login, email/username changes, and examples containing extra privilege-related fields. Page 3 has limited readable text in the preview.

Educational use only. Avoid unauthorized privilege testing.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Password Reset Security Checklist

Educational Bug Bounty Series • Authorized Security Research Only

■ Password Reset & Account Recovery — Security Checklist

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Use controlled accounts and inboxes. Never attempt to reset another person's password or reuse their tokens.*

■ Executive Summary

Password-reset flows are sensitive authentication mechanisms. The supplied two-page checklist describes risks involving token validation, Host header injection, referrer leakage, HTML injection in email, company-invitation flows, token removal or mutation, null/old/oversized tokens, request-method and content-type variations, response manipulation, cross-domain token use, and race conditions.

■ Core Testing Methodology

1. Use a dedicated test account and controlled email inbox to request a reset.
2. Verify reset tokens are cryptographically random, sufficiently long, single-use, and short-lived.
3. Confirm tokens are bound to the intended account and purpose.
4. Ensure generated links use a configured canonical domain, not untrusted Host or forwarding headers.
5. Check that reset tokens are not leaked through referrer headers, analytics, logs, or response bodies.
6. Verify malformed, missing, expired, reused, null-like, or altered tokens fail safely.
7. Check that HTTP method and content-type changes do not bypass server-side validation.
8. Ensure responses do not reveal whether an account exists.
9. Verify reset requests are rate-limited and monitored.
10. Review concurrent reset attempts and token invalidation behavior using the test account.
11. Review invitation and company-email flows separately; do not add unexpected password fields to real users' requests.
12. Document the expected behavior and retest remediation.

■ High-Value Bug Bounty Tips & Edge Cases

- **Token binding:** A token generated for one account must not work for another account or domain.
- **Host header:** Reset links should use a configured canonical origin.
- **Referrer leakage:** Sensitive tokens in URLs can leak to third-party resources; apply a restrictive Referrer-Policy and avoid loading third-party content on reset pages.
- **Token mutations:** Empty, altered, expired, reused, or malformed tokens should be rejected consistently.
- **Old token arrays:** Previous tokens must not remain valid after successful reset unless explicitly designed and safely constrained.
- **Response manipulation:** Client-side changes to a response must not change server-side verification state.
- **Cross-domain use:** The source notes multi-domain applications; tokens should be bound to the intended application and reset purpose.
- **Race conditions:** Concurrent requests should not allow a token to be reused or a password reset to be applied unpredictably.
- **SQL injection and parser variations:** Treat these as code-review/testing categories; do not use destructive payloads on production.

- **Source limitation:** The checklist references an external account-takeover article and contains several terse examples. This rewrite summarizes categories without treating each as a verified bypass.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Test account: [Controlled-Test-Account]
Token expiry enforced: [Yes/No]
Token single-use: [Yes/No]
Token bound to account/domain: [Yes/No]
Canonical reset URL: [Yes/No]
Token exposed in referrer/logs: [Yes/No]
Rate limiting: [Yes/No]
Evidence: [Redacted]
```

Example of a safe reset request shape:

```
POST /account/password-reset HTTP/1.1
Host: [Configured-Canonical-Domain]
Content-Type: application/json

{
  "email": "[CONTROLLED-TEST-EMAIL]",
  "token": "[REDACTED-RESET-TOKEN]",
  "new_password": "[TEST-ONLY-PASSWORD]"
}
```

- [ ] Tokens are unpredictable, expiring, single-use, and account-bound.
- [ ] Reset URLs use a configured origin.
- [ ] Responses avoid account enumeration.
- [ ] Tokens are not logged or sent to third parties.
- [ ] Rate limits and audit logging are in place.

■■ Mitigation & Remediation

Use high-entropy single-use tokens with short expiry; bind them to account and purpose; invalidate them after use or credential change; construct links from a trusted canonical origin; prevent token leakage through referrers and logs; rate-limit requests; use generic responses; and test concurrency and cross-domain boundaries.

■ Source Notes

Based on the supplied two-page *reset pass checklist.pdf*. Page 1 lists token use, Host header injection, HTML injection, referrer leakage, company-email invitation behavior, CRLF, email injection, and token removal. Page 2 lists zero/null and old tokens, SQLi category, method/content-type changes, response manipulation, long tokens, cross-domain token usage, response-body leakage, Unicode/race-condition notes, and email-domain variations.

Educational use only. Use only controlled accounts and inboxes.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Recon Target Based Strategies

Educational Bug Bounty Series • Authorized Security Research Only

■ Reconnaissance Strategies — Target-Based Scope

Contributed by Cyber_Spy.404 | Bug Bounty Notes

Authorization rule: *The program's written scope always overrides a general reconnaissance checklist. Do not scan related companies, IP ranges, or subdomains unless they are explicitly in scope.*

■ Executive Summary

The source HTML, *Recon Target Based.html*, organizes reconnaissance into **small**, **medium**, and **large** scope. It scales the breadth of discovery according to the scope definition, from a single URL to organization-wide assets. The source lists directory enumeration, technology fingerprinting, ports, parameters, historical URLs, JavaScript review, cloud storage, dorking, and vulnerability checks.

■ Core Testing Methodology

1. Small Scope Reconnaissance

Source definition: Specific URLs only, often staging/development/testing or a single URL.

- [ ] Directory enumeration
- [ ] Technology fingerprinting
- [ ] Port scanning, only where permitted
- [ ] Parameter fuzzing, within program limits
- [ ] Wayback/history review
- [ ] Known-vulnerability review
- [ ] Hardcoded information in JavaScript
- [ ] Domain-specific GitHub and Google search
- [ ] Broken-link hijacking assessment
- [ ] Data-breach exposure review without using leaked credentials
- [ ] Misconfigured cloud-storage review

2. Medium Scope Reconnaissance

Source definition: Usually wildcard scope where all subdomains are included.

- [ ] Subdomain enumeration
- [ ] Subdomain takeover checks
- [ ] Probing and technology fingerprinting
- [ ] Port scanning within scope
- [ ] Known-vulnerability review
- [ ] Template-based scanning (the source names Nuclei/“Jeales”)
- [ ] Misconfigured cloud-storage review
- [ ] Broken-link hijacking assessment
- [ ] Directory enumeration
- [ ] JavaScript review

- [ ] GitHub reconnaissance
- [ ] Google dorking
- [ ] Data-breach exposure review
- [ ] Parameter fuzzing
- [ ] Internet search engines (the source names Shodan, Censys, Spyse)
- [ ] IP-range enumeration only if explicitly in scope
- [ ] Wayback/history review
- [ ] Pattern extraction with GF for categories such as XSS/SSRF, followed by safe manual validation
- [ ] Heartbleed exposure review where applicable and authorized
- [ ] General security-misconfiguration review

3. Large Scope Reconnaissance

Source definition: Organization-related assets, including child companies, subdomains, or labelled assets, are considered in scope **only when the program explicitly says so**.

- [ ] Track application signatures and crawl authorized targets
- [ ] Subsidiary/acquisition enumeration, bounded by written scope
- [ ] Reverse lookup
- [ ] ASN and IP-space enumeration and service identification, only for authorized ranges
- [ ] Subdomain enumeration and takeover checks
- [ ] Probing and technology fingerprinting
- [ ] Port scanning
- [ ] Known-vulnerability review
- [ ] Template-based scanning
- [ ] Cloud-storage and broken-link-hijacking checks
- [ ] Directory enumeration and JavaScript review
- [ ] GitHub and Google reconnaissance
- [ ] Data-breach exposure review without credential use
- [ ] Parameter fuzzing
- [ ] Search-engine discovery (Shodan/Censys/other sources)
- [ ] IP-range enumeration only when included in scope
- [ ] Wayback/history review
- [ ] Pattern extraction for relevant vulnerability classes
- [ ] Heartbleed and general security-misconfiguration review
- [ ] Consider other network/web reconnaissance vectors only when authorized

■ High-Value Tips & Edge Cases

- A wildcard domain does **not** automatically include the organization's IP ranges, subsidiaries, third-party SaaS, or cloud tenants.
- Passive discovery is not the same as permission for active scanning.
- A scanner finding is a lead, not a confirmed vulnerability.
- Check scope exclusions, request limits, safe-harbor terms, and rules for automated tools before testing.
- Broken-link hijacking, subdomain takeover, and cloud-storage issues can affect real users; use non-invasive verification and stop before claiming or taking over a resource.

- Data-breach sources may contain credentials or personal data. Do not log in, reuse secrets, or download unrelated private information.

■■ Educational Checklist & Safe Examples

```
Program: [Program Name]
Asset: [Exact In-Scope Asset]
Scope type: [Single URL / Wildcard / Organization-defined]
Active scanning permitted: [Yes / No / Limited]
Rate limit: [Program-defined]
Recon source: [Passive / Active]
Observation: [Non-sensitive summary]
Status: [Lead / Confirmed / Not Reproduced]
Evidence: [Redacted]
```

- [ ] Written scope saved and reviewed
- [ ] Exclusions and rate limits noted
- [ ] Assets verified against the program scope
- [ ] Passive and active steps distinguished
- [ ] Findings manually validated
- [ ] Sensitive information not accessed or copied
- [ ] Report contains minimal, redacted evidence

■■ Mitigation & Remediation

For defenders, maintain an authoritative asset inventory, remove abandoned DNS/service records, restrict exposed services, protect cloud storage, avoid secrets in client-side code, patch exposed services, and continuously monitor assets. Define clear scope and safe-testing rules for bounty programs.

■ Source Notes

Based on *Recon Target Based.html*. The source defines three scope levels and lists the reconnaissance activities under each. This guide preserves those categories while clarifying that organization-related assets and active tests require explicit program authorization. The source's spelling “Jeales” is retained in the checklist note; the HTML does not define that tool.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Register Vulnerability Testing

Educational Bug Bounty Series • Authorized Security Research Only

■ Registration Vulnerability Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Use only accounts you control and authorized test environments. Do not manipulate real users' accounts or exploit registration flaws on third-party systems.*

■ Executive Summary

Registration weaknesses can allow duplicate-account confusion, unsafe profile paths, weak password policies, or incorrect verification behavior. The supplied one-page checklist lists duplicate registration with case variations, missing name/password validation, absent rate limits, XSS in registration fields, weak email verification, disposable email acceptance, HTTP registration forms, weak passwords, recovery issues, and path overwrite concerns.

■ Core Testing Methodology

1. Create a test account with an email address you control and record expected behavior.
2. Check whether registration prevents duplicate accounts using the same email, including case normalization.
3. Verify username and password fields have clear required-field, length, and format validation.
4. Check whether repeated sign-up attempts are rate-limited.
5. Use harmless markers in username/email fields to verify output encoding; do not use data-exfiltration payloads.
6. Confirm email verification requires a server-validated, single-use token and cannot be bypassed by changing client-side responses.
7. Verify disposable-email policy is explicit and consistently enforced if the service has such a policy.
8. Confirm registration pages and submissions use HTTPS.
9. Review password policy, password reuse controls where applicable, and recovery flows.
10. Check whether profile routes safely handle reserved names and cannot overwrite system paths.
11. Document the expected policy, observed behavior, and minimal redacted evidence.

■ High-Value Bug Bounty Tips & Edge Cases

- **Case normalization:** The source suggests registering the same email with case variations. Email canonicalization must be consistent and must not create ambiguous identities.
- **Long inputs:** The source notes long password values leading to a possible server error. Validation should return a controlled client error rather than HTTP 500.
- **Rate limits:** Registration should have abuse controls appropriate to the product and threat model.
- **XSS:** The source notes username/email reflection; test with a harmless marker and inspect encoding.
- **Email verification:** The checklist mentions response/status manipulation. Client-side response changes must never establish verification; only server-side token validation should.
- **Transport security:** Registration should not submit credentials or personal data over plain HTTP.
- **Weak passwords:** The source suggests checking simple passwords and password-recovery behavior. Do not test credentials against other users.
- **Path overwrite:** The source mentions reserved system filenames/profile paths, but does not give a complete scenario. Treat this as a review item, not a proven exploit.

■■ Educational Checklists & Payloads

```
Target: [Your-Domain]
Test account: [Controlled-Test-Email]
Duplicate registration blocked: [Yes/No]
Case normalization consistent: [Yes/No]
Long input handled safely: [Yes/No]
Registration rate limit present: [Yes/No]
Verification token single-use: [Yes/No]
HTTPS enforced: [Yes/No]
Evidence: [Redacted]
```

Safe output-encoding marker:

```
registration-test-[UNIQUE-ID]
```

- [ ] Required fields and length limits are enforced server-side.
- [ ] Duplicate identities are handled consistently.
- [ ] User-controlled values are output-encoded.
- [ ] Verification is server-side and token-based.
- [ ] Passwords are stored using a modern password-hashing function.
- [ ] Rate limits and abuse detection are enabled.
- [ ] Registration uses HTTPS.

■■ Mitigation & Remediation

Normalize identity values consistently, enforce server-side validation and rate limits, use secure password hashing and password policy, require single-use expiring verification tokens, encode output by context, enforce HTTPS, and protect profile routes from reserved-name/path conflicts.

■ Source Notes

Based on the supplied one-page *register vuln.pdf*. It lists duplicate registration, long-input errors, no signup rate limit, XSS in username/email, verification response/status manipulation, disposable email, HTTP registration, weak passwords, recovery-link behavior, and path overwrite. The source does not provide a full reproduction for every item.

Educational use only. Test only accounts you control.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Remote Code Execution Classes

Educational Bug Bounty Series • Authorized Security Research Only

■ Remote Code Execution (RCE) — Vulnerability Classes & Defensive Review

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Review authorized code/configurations or use an isolated lab. Do not run exploit payloads, execute commands on third-party systems, or access internal services.*

■ Executive Summary

Remote Code Execution (RCE) occurs when an attacker causes a server to execute unintended code. The supplied two-page *rce.pdf* is a reading list covering dependency confusion, file uploads, SQL injection, LFI, SSRF, XXE, command injection, insecure deserialization, and server-side template injection.

■ Core Testing Methodology

1. Inventory uploads, dependency pipelines, database interfaces, file access, URL fetchers, XML parsers, deserializers, templates, and process calls.
2. Review package sources, namespace ownership, lockfiles, and CI/CD integrity.
3. Verify uploaded files cannot execute and are processed safely.
4. Check database operations use parameterized queries.
5. Constrain file paths and server-side network destinations.
6. Disable unsafe XML external entities.
7. Review command execution and avoid shell-string concatenation.
8. Review deserialization and template handling in code or an isolated lab.
9. Prefer configuration review, unit tests, and harmless canary markers.
10. Document the unsafe code path and retest remediation.

■ High-Value Bug Bounty Tips & Edge Cases

• Dependency confusion: use trusted registries, private namespaces, and pinned dependencies.

• Uploads: validate content and prevent execution from upload directories.

• SQL injection: parameterize queries and limit database privileges.

• LFI/SSRF: constrain paths and outbound requests; do not retrieve secrets or probe internal services.

• XXE: disable external entities and external DTD loading.

• Command injection: prefer language-native APIs.

• Deserialization: avoid untrusted native-object deserialization.

• SSTI: keep templates trusted and treat user input as data.

• The PDF is a list of links and categories, not a reproducible exploit guide.

■■ Educational Checklists & Safe Examples

```
Component: [Upload / Dependency / Query / URL Fetch / XML / Template]
Input source: [User-Controlled Field]
Suspected unsafe operation: [Code Path / Configuration]
Evidence: [Code Review / Sanitized Test]
Remediation: [Specific Change]
```

- [ ] Dependencies are pinned and trusted.
- [ ] Uploads cannot execute.
- [ ] Queries are parameterized.
- [ ] File and network access are constrained.
- [ ] XML external entities are disabled.
- [ ] Shell commands and unsafe deserialization are avoided.
- [ ] User input is not evaluated as template code.

■■ Mitigation & Remediation

Use trusted dependency registries and lockfiles, isolate uploads, parameterize queries, constrain file/network access, disable unsafe XML features, replace shell calls with native APIs, avoid unsafe deserialization, keep templates trusted, and apply least privilege and regression tests.

■ Source Notes

Based on the supplied two-page *rce.pdf*. Page 1 lists dependency confusion, file upload, SQL injection, LFI, SSRF, XXE, and command injection; page 2 lists insecure deserialization and SSTI. The source contains references rather than detailed procedures.

Educational use only. RCE testing requires explicit authorization and isolation.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Shodan Dorking Safe Guide

Educational Bug Bounty Series • Authorized Security Research Only

■ Shodan Dorking — Authorized Asset Discovery

Contributed by Cyber_Spy.404 | Bug Bounty Notes

Critical safety note: *Internet-wide search results can include hospitals, industrial controls, cameras, routers, remote-access services, and other sensitive systems. Search and test only assets explicitly authorized by the program; never attempt access or interaction with critical infrastructure.*

■ Executive Summary

The source HTML, *shodan.html*, groups Shodan filters and examples into basic filters, web services, databases, industrial control systems (ICS), remote desktop, network infrastructure, Outlook Web Access, NAS, webcams, printers/copiers, home devices, and miscellaneous systems. Many source examples identify exposed or weakly protected services. This guide retains the categories and basic filter concepts while omitting actionable query strings that target vulnerable critical infrastructure or unauthenticated devices.

■ Core Testing Methodology

1. Confirm the exact in-scope domain, IP, ASN, and any excluded ranges.
2. Begin with the least intrusive search filters and use them only to locate assets already authorized for assessment.
3. Treat banners and product labels as indicators, not proof of a vulnerability or current exposure.
4. Verify versions and exposure through vendor documentation or owner-approved checks.
5. Do not log in, interact with control panels, change settings, send commands, or access data.
6. For critical infrastructure and medical/industrial devices, report passive observations and seek program guidance before any further validation.
7. Keep screenshots and output redacted; do not publish device locations, credentials, or personal information.

■ High-Value Tips & Edge Cases

Basic filter concepts present in the source

- `hostname:` — hostname filter
- `net:` — IP address or CIDR network
- `org:` — organization label
- `asn:` — Autonomous System Number
- `country:` / `city:` / `geo:` — location-related filters
- `port:` — port number
- `product:` / `os:` / `device:` — product, operating system, or device classification
- `server:` — server banner
- `ssl.cert.*` — certificate attributes
- `before:` / `after:` — time filters

Safe template: use a verified, authorized asset value rather than a broad city, country, device, or product query.

```
hostname:YOUR-AUTHORIZED-DOMAIN.example
net:YOUR-APPROVED-CIDR
org:"YOUR-ORGANIZATION"
```

These are templates; replace them only with values explicitly included in the program scope.

Source categories preserved

- **Basic Shodan filters:** city, country, geographic coordinates, hostname, network, organization, ASN, OS, port, time, TLS certificates, device type, product, CPE, server, SSH fingerprints.
- **Web-related services:** VPN/web gateways and certificate/file-index indicators.
- **Databases:** MySQL, MongoDB, Elasticsearch, Memcached, CouchDB, PostgreSQL, Riak, Redis, Cassandra.
- **Industrial control systems (ICS):** billboards, fuel systems, traffic systems, voting systems, energy generation/meters, building automation, access controllers, rail systems, and industrial protocols including Modbus, BACnet, DNP3, EtherNet/IP, S7, and others.
- **Remote desktop:** VNC and RDP.
- **Network infrastructure:** routers, Redis, Citrix, Kubernetes/Docker-related dashboards/APIs, Jenkins, DNS, Telnet, video conferencing, management interfaces, and known historical CVEs.
- **OWA and collaboration:** Exchange and Lync/Skype for Business.
- **NAS and file services:** SMB, FTP, NAS products, media servers, and attached storage.
- **Webcams:** camera/webcam products and surveillance systems.
- **Printers/copiers:** HP, Xerox, Epson, Canon.
- **Home devices:** audio receivers, AirPlay, smart TVs/Chromecast, smart-home controllers.
- **Miscellaneous:** 3D-printer controllers, mining systems, directory listings, WordPress, game servers, and other internet-facing services.

■■ Educational Checklist & Safe Examples

```
Program: [Program Name]
Authorized asset: [Exact Host/IP/CIDR]
Shodan filter: [Scope-bound filter]
Observation: [Banner/version, no sensitive data]
Vendor advisory checked: [Yes/No]
Active validation permitted: [Yes/No]
Finding status: [Lead / Confirmed / Inconclusive]
Evidence: [Redacted]
```

- [ ] Exact scope confirmed
- [ ] No broad searches for critical infrastructure or private devices
- [ ] No authentication attempts or interaction with exposed systems
- [ ] Version/banner evidence treated cautiously
- [ ] Vendor advisories checked
- [ ] Evidence redacted and minimally retained

■■ Mitigation & Remediation

Remove unnecessary public exposure, restrict management interfaces by network policy, require strong authentication and MFA, patch firmware and services, disable unused protocols, segment OT/ICS from the public internet, monitor asset exposure, and follow vendor-specific hardening guidance. Critical infrastructure should use defense-in-depth and strict network segmentation.

■ Source Notes

Based on *shodan.html*. The source has sections for basic filters, web services, databases, ICS, remote desktop, network infrastructure, OWA, NAS, webcams, printers/copiers, home devices, and miscellaneous systems. It includes numerous searches for unauthenticated or sensitive devices; the operational query strings are intentionally not reproduced. The original examples may be outdated and should not be treated as proof of present-day exposure.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Symfony Security Review

Educational Bug Bounty Series • Authorized Security Research Only

■ Symfony Security Review

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Review only authorized Symfony applications. Do not retrieve configuration secrets, execute code, or access private profiler data.*

■ Executive Summary

Misconfigured Symfony deployments may expose development tools, profiler endpoints, configuration files, or sensitive debug information. The supplied one-page checklist references Symfony RCE research, the `eos` tool, sensitive-path examples, secret-fragment research, template-checklist files, and a Symfony wordlist for fuzzing.

■ Core Testing Methodology

1. Confirm the target is in scope and identify the Symfony version where safely possible.
2. Review deployment configuration to confirm production mode is enabled and debug features are disabled.
3. Check, with minimal requests, whether development profiler or debug routes are publicly accessible.
4. Verify configuration and test files are not served by the web server.
5. Check that error pages do not disclose paths, environment variables, secrets, or stack traces.
6. Review templates, routing, and user-controlled input handling through code/configuration review or a staging environment.
7. Compare the installed version and dependencies with official security advisories.
8. Avoid secret-fragment or RCE exploitation on live targets; use isolated labs for validation.
9. Record minimal redacted evidence and retest.

■ High-Value Bug Bounty Tips & Edge Cases

- The source lists profiler/debug-style paths and configuration/test-file examples as sensitive exposure categories. These are checks, not confirmation of exposure.
- Development tools must not be available in production.
- Template injection and RCE are high-impact classes; assess safely via code review or isolated testing.
- The source references `eos`, secret-fragment research, template checklists, and a Symfony fuzzing wordlist; it does not provide a complete verified exploit procedure.
- Fuzzing can create load and noisy results; obtain explicit authorization and use a conservative rate.

■■ Educational Checklists & Safe Examples

```
Application: [Authorized-Symfony-Host]
Environment: [Production / Staging]
Version: [Verified-Version]
Profiler/debug routes exposed: [Yes/No/Inconclusive]
Config/test files publicly served: [Yes/No/Inconclusive]
Sensitive errors: [Yes/No]
Evidence: [Redacted]
```

- [ ] Production debug mode is disabled.
- [ ] Profiler and development routes are restricted.
- [ ] Config, secrets, and test files are outside public web root.
- [ ] Templates treat user input as data.

- [ ] Dependencies are patched.
- [ ] Error pages do not expose internals.

■■ Mitigation & Remediation

Set production environment and disable debug mode, restrict profiler routes, keep configuration and secrets outside public directories, use least-privilege file permissions, patch Symfony and dependencies, avoid unsafe template evaluation, and add deployment checks to prevent development artifacts from shipping.

■ Source Notes

Based on the supplied one-page *Symfony.pdf*. It lists RCE research, an `eos` tool reference, profiler/configuration path categories, secret-fragment research, template-checklist references, and a Symfony wordlist. No exploit result or affected version is established by the source.

Educational use only. Test within authorized scope.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

Vulnerability Checklist Index

Educational Bug Bounty Series • Authorized Security Research Only

■ Vulnerability Checklist Index

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

■ Executive Summary

The supplied *Vuln Checklist.pdf* is an index of topics in an existing vulnerability-checklist repository. It lists 27 categories spanning authentication, authorization, injection, configuration, API security, framework-specific review, and common web vulnerabilities. This file preserves the index structure rather than reproducing the linked repository's contents.

■ Core Testing Methodology

1. Confirm the target and permitted test methods.
2. Choose the checklist matching the application's technology and feature.
3. Establish expected security behavior before testing.
4. Use controlled accounts and non-sensitive test data.
5. Follow each checklist in scope and record results.
6. Report confirmed issues with minimal redacted evidence.
7. Retest fixes and update the checklist status.

■ High-Value Bug Bounty Tips & Bypasses

The source index lists these categories:

1. AEM misconfiguration
2. Authentication
3. IDOR
4. Business logic
5. Jira vulnerability
6. Registration vulnerability
7. 2FA bypassing
8. Admin panel
9. EXIF vulnerability
10. Cookie attack
11. Password reset attack
12. Account takeover checklist
13. HTTP 403 bypass checklist
14. Tips from Twitter
15. Tips from Twitter, part 2
16. SQL injection
17. XSS
18. File upload

19. Rate-limit bypass
20. JSON attack
21. CSRF
22. RCE
23. API authorization
24. API authentication
25. Mass assignment
26. Django checklist
27. Symfony hacking

The source provides links to checklist files but does not itself contain their detailed procedures.

■■ Educational Checklists & Safe Examples

```
Target: [Your-Domain]
Checklist category: [Category]
Scope verified: [Yes/No]
Test data: [Controlled / Synthetic]
Finding: [Confirmed / Not Reproduced / Inconclusive]
Evidence: [Redacted]
Remediation retested: [Yes/No]
```

- [ ] Scope checked.
- [ ] Correct checklist selected.
- [ ] Controlled test data used.
- [ ] No secrets or third-party data copied.
- [ ] Evidence is redacted.
- [ ] Remediation retested.

■■ Mitigation & Remediation

The index does not provide category-specific fixes. In general, use the relevant vendor/framework guidance and implement server-side authentication and authorization, strict input validation, secure configuration, safe output encoding, rate limiting, dependency patching, and regression tests as appropriate to the confirmed finding.

■ Source Notes

Based on the supplied two-page *Vuln Checklist.pdf*. Pages 1–2 list 27 linked checklist categories and identify the referenced repository. The PDF is an index only; detailed procedures and remediation steps are not included in the source.

Educational use only. Use the original checklist references responsibly.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

XSS Cheat Sheet Basic and Advanced

Educational Bug Bounty Series • Authorized Security Research Only

■ XSS Cheat Sheet — Basic Concepts & Defensive Testing

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Use only authorized environments and harmless markers. Do not use payloads to steal cookies, tokens, source code, or personal information.*

■ Executive Summary

Cross-Site Scripting (XSS) is an injection flaw that causes a browser to interpret untrusted input as active content. The supplied seven-page *xss.pdf* describes reflected, stored, and DOM-based XSS; input reflection contexts; multiple reflections; filenames and metadata; SVG/XML/Markdown; encoding; and WAF-specific notes.

■ Core Testing Methodology

1. Identify input sources and where their values are rendered.
2. Insert a unique harmless marker and trace it through the response and DOM.
3. Classify the context: HTML text, attribute, script string, URL, CSS, comment, or DOM sink.
4. Check whether output is encoded according to the destination context.
5. Review reflected, stored, and DOM-based flows separately.
6. Test stored content only in an authorized test account and avoid exposing it to other users.
7. Review upload filenames and metadata with synthetic files.
8. Check Markdown, SVG, XML, and other formats only where the application intentionally supports them.
9. Use scanners as supporting evidence, then manually verify findings.
10. Report a minimal reproduction and retest the fix.

■ High-Value Bug Bounty Tips & Bypasses

- **Reflected XSS:** Input is returned in the response; context and encoding determine risk.
- **Stored XSS:** Input is persisted and later rendered; test with controlled content.
- **DOM-based XSS:** Client-side JavaScript may route untrusted data into unsafe DOM sinks.
- **HTML contexts:** Comments, tags, and attributes require different handling.
- **JavaScript contexts:** String delimiters, escaping, and template construction can affect parsing; avoid executable bypass strings in public notes.
- **Multiple reflections:** One value may be inserted into more than one context.
- **File metadata:** Filenames and EXIF fields can become XSS sources if rendered unsafely.
- **SVG/XML/Markdown:** Active features differ by parser and serving context; sanitize or disable them where not required.
- **Encoding:** Repeated decoding can create inconsistencies; define canonicalization and encode at output.
- **WAF bypasses:** The source lists mixed-case, encoded, and vendor-specific variants. A WAF is not a primary fix, and these strings are intentionally omitted.
- **Source limitation:** The PDF contains many active script payloads and WAF bypasses. This guide preserves the concepts without reproducing the exploit catalogue.

■■ Educational Checklists & Safe Examples

Harmless marker:

```
XSS_TEST_12345
```

Example safe output:

```
<p>Search term: XSS_TEST_12345</p>
```

- [ ] Input source and output context are identified.
- [ ] Context-aware output encoding is applied.
- [ ] Untrusted input is not evaluated as code.
- [ ] DOM code avoids unsafe HTML-string insertion.
- [ ] Stored content is tested only with controlled accounts.
- [ ] Upload metadata and active file formats are handled safely.
- [ ] CSP is used as defense in depth.

■■ Mitigation & Remediation

Use context-aware output encoding, safe template defaults, maintained HTML sanitization libraries when HTML is a product requirement, safe DOM APIs, and a restrictive Content Security Policy. Serve untrusted files from an isolated origin and add regression tests for every confirmed reflection context.

■ Source Notes

Based on the supplied seven-page *xss.pdf*. Page 1 introduces reflected, stored, and DOM-based XSS and basic reflection contexts. Pages 2–4 cover JavaScript-string contexts, escaping, multiple reflections, comments, and file-upload filename/metadata cases. Page 5 covers SVG, Markdown, XML, encoding, and validation variants. Pages 6–7 list WAF-specific bypass examples for multiple providers. Active payloads are intentionally not reproduced.

Educational use only. Validate safely and disclose responsibly.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.

XSS Testing Methods

Educational Bug Bounty Series • Authorized Security Research Only

■ Cross-Site Scripting (XSS) — Testing Methods

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope: *Test only authorized applications. Use harmless markers and controlled browsers. Never steal cookies, tokens, source code, or user data.*

■ Executive Summary

Cross-Site Scripting (XSS) occurs when untrusted input reaches a browser context in a way that allows unintended script execution. The supplied *xss 2.pdf* collects scanner references, endpoint-discovery approaches, reflection checks, file-upload cases, WAF notes, and payload examples. This rewrite focuses on safe validation and defense rather than executable bypass strings.

■ Core Testing Methodology

1. Identify in-scope input points in query parameters, forms, JSON APIs, filenames, metadata, and client-side DOM flows.
2. Establish a baseline and insert a unique harmless marker such as `XSS_TEST_12345`.
3. Inspect the response and rendered DOM to see whether the marker is reflected and in which context.
4. Distinguish HTML text, attributes, JavaScript strings, URLs, CSS, SVG, and DOM-based sinks.
5. Check whether output is encoded for its exact context and whether dangerous DOM APIs are used.
6. Review file uploads and metadata only with synthetic files you own.
7. Use automated scanners only within scope, with conservative request rates and reviewed output.
8. Treat WAF behavior as a signal, not proof; verify the application's own encoding and sanitization.
9. Record a minimal reproduction without cookie theft, outbound callbacks, or access to other users' sessions.
10. Retest after remediation.

■ High-Value Bug Bounty Tips & Bypasses

- **Context matters:** A marker in HTML text behaves differently from one in an attribute, script block, URL, or comment.
- **Multiple reflections:** One value may appear in several contexts on the same page; review each separately.
- **DOM-based XSS:** Data may flow through JavaScript without appearing in the original HTML response.
- **File-upload XSS:** Filenames, image metadata, SVG, Markdown, and XML may be rendered by different components.
- **WAF behavior:** The source lists vendor-specific WAF bypass strings. WAFs are not a substitute for correct output encoding; this guide omits the bypass payload catalogue.
- **Scanner limitations:** Automated tools can miss context-specific issues or report false positives. Manually verify with harmless markers.
- **Endpoint discovery:** JavaScript files and source maps may reveal routes or parameter names; stay in scope and do not harvest secrets.
- **Source limitation:** The source contains numerous executable XSS and cookie/source exfiltration payloads. They are intentionally not reproduced.

■■ Educational Checklists & Safe Examples

Harmless marker:

```
XSS_TEST_12345
```

Example reflection review:

```
<p>Search term: XSS_TEST_12345</p>
```

- [ ] Marker location and output context are documented.
- [ ] HTML text is encoded.
- [ ] Attribute values are safely quoted and encoded.
- [ ] User input is not concatenated into executable JavaScript.
- [ ] DOM sinks use safe APIs such as `textContent` where suitable.
- [ ] Uploads and metadata are sanitized or served as downloads.
- [ ] Reports do not contain stolen tokens or user data.

■■ Mitigation & Remediation

Use context-aware output encoding, safe templating, strict input handling, and maintained sanitizers for intentionally supported HTML. Avoid unsafe DOM APIs such as assigning untrusted strings to `innerHTML`; prefer `textContent` and safe DOM construction. Apply a restrictive Content Security Policy as defense in depth, and isolate untrusted uploaded content on a separate origin.

■ Source Notes

Based on the supplied three-page *xss 2.pdf*. Page 1 references XSS scanners, URL/parameter discovery, and endpoint fuzzing. Page 2 contains automated workflow notes and executable payload examples involving document/window APIs, callbacks, and input tags. Page 3 contains WAF-bypass categories and obfuscated payloads. These operational payloads are intentionally replaced with safe markers and defensive review steps.

Educational use only. Stay within scope and protect user data.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.